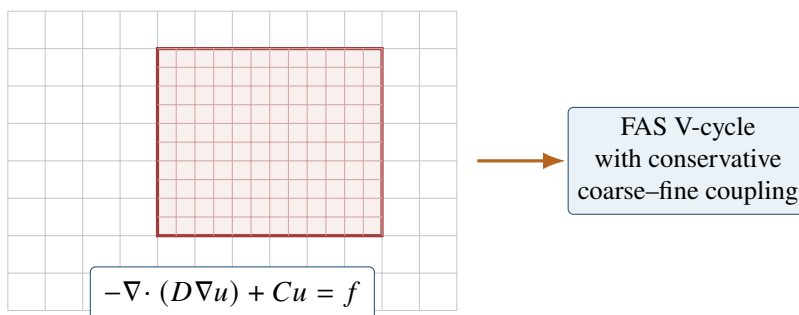


A Block-Structured Adaptive Multigrid (BSAM) Solver

Mathematical and Code Documentation for the Supplied MATLAB Implementation



Textbook-style guide for readers with a background in numerical analysis and multigrid methods

June 2026

Preface

This document explains the supplied MATLAB code as an integrated numerical method, not merely as an API catalog. It develops the cell-centered flux discretization, boundary closures, block-structured hierarchy, transfer operators, adaptive full approximation storage (AFAS) cycle, conservative interface correction, tagging and clustering, diagnostics, and implementation data structures. A dedicated dry-run chapter follows one solve from the bottom direct solve through the first adaptive refinement and the first composite V-cycle. mememememe

The code is an implementation of the mass-conservative adaptive FAS construction associated with Feng, Guo, Lowengrub, and Wise. Paper equation numbers already present in source comments are retained where useful, but the supplied code is treated as the authoritative description of actual behavior. In particular, this guide documents implementation details that are easy to miss in a mathematical presentation: the distinction between sub-root and adaptive levels, precomputed gather/scatter descriptors, exact boundary folding in the smoother, transient parent-coefficient materialization, global red-black parity across patches, and the life cycle of the QC restriction buffer.

The source uses the prefix `eb`; diagnostic messages use the name “`ebcam`.” This guide uses *BSAM* as the project name and *AFAS* for the adaptive full-approximation-storage algorithm.

How to read this document. Readers who want the overall picture first can go straight to the dry run in [chapter 8](#), which follows one solve end to end, and then return to [chapters 2 to 6](#) for the individual components it exercises. Readers who only want to run the solver can start with [chapter 10](#). [Chapter C](#) collects the translation between the mathematical symbols used here and the code arrays, and is a useful companion throughout.

Contents

Preface	i
1 Orientation: What the Solver Does	1
1.1 Problem class	1
1.2 Top-level execution path	1
1.3 The three nested loops	2
1.4 Source modules by responsibility	3
2 Cell-Centered Discretization	4
2.1 Level grid and cell centers	4
2.2 Face-centered coefficients	4
2.3 Flux-form operator	5
2.4 Source representation	5
2.5 Operator properties and assumptions	6
3 Hierarchy, Patches, and the Data Model	7
3.1 Sub-root levels versus adaptive levels	7
3.2 How the uniform ladder is selected	7
3.3 Patches, coverage, and visible cells	8
3.4 Patch lineage and bookkeeping	9
3.5 The hierarchy structure H	9
3.6 Per-level and per-patch fields	9
3.7 Precomputed communication descriptors	11
4 Ghost Cells and Boundary Closures	12
4.1 Why ghost cells carry the coupling	12
4.2 Canonical fill order	12
4.3 Quadratic Dirichlet closure	12
4.4 Neumann closure and sign convention	13

4.5	Exact boundary treatment in red–black relaxation	13
4.6	Coarse–fine interpolation	14
4.6.1	Quadratic interpolation (QIF)	16
4.6.2	Linear interpolation (LIF)	16
4.7	Same-level exchange and global consistency	17
5	Interlevel Transfer and Coefficient Compatibility	18
5.1	Restriction	18
5.2	The padded parent window	18
5.3	Bilinear prolongation	19
5.4	Coefficient fill-down	19
5.5	Transient parent coefficients	19
5.6	Transfer modes in ebTransfer	20
5.7	Preconditions on QC	20
6	Adaptive FAS Multigrid and Conservative Interfaces	21
6.1	Why FAS is used for a linear equation	21
6.2	The FAS coarse equation	21
6.3	Why ghost interpolation alone is not conservative	21
6.4	Flux-balance correction	22
6.5	Recursive V-cycle	22
6.6	Red–black Gauss–Seidel smoother	24
6.7	The level-1 direct solve	24
6.8	One V-cycle as a force lifecycle	24
7	Adaptive Refinement	25
7.1	Two refinement workflows	25
7.1.1	Solution-driven indicators	25
7.1.2	Geometry-driven indicator	25
7.2	Undivided-Laplacian indicator	25
7.3	Relative truncation-error indicator	26
7.4	User tag function	26
7.5	Buffering and proper nesting	26
7.6	Signature-based box clustering	26
7.7	Appending a new level	27
7.8	The adaptive-stage stopping rule	28

8	A Dry Run: From the Bottom Solve to the First Adaptive Level	29
8.1	Purpose of the tour	29
8.2	Stage 0: hierarchy construction and bottom factorization	29
8.3	Stage 1, first V-cycle: descend from the root	30
8.3.1	Level 3 to level 2	30
8.3.2	Level 2 to level 1	31
8.4	The bottom event: direct level-1 solve	31
8.5	Ascend from level 1 to the root	31
8.6	The first tagging pass	31
8.7	Creating level 4	32
8.8	Initializing the new fine patch	32
8.9	The first composite V-cycle on levels 1–4	33
8.9.1	Pre-smooth the fine patch	33
8.9.2	Restrict the fine patch	33
8.9.3	Load the root FAS equation	33
8.9.4	Recurse through root and sub-root levels	34
8.9.5	Correct level 4	34
8.9.6	Synchronize and measure	34
8.10	State table for the adaptive descent/ascent	35
8.11	What changes at a second refinement	35
9	Convergence Control and Diagnostics	36
9.1	Composite residual	36
9.2	Solve-stage driver	36
9.3	Error norms	37
9.4	Composite mean and final gauge	37
9.5	Discrete mass-conservation check	38
9.6	Sampling and plotting	38
9.7	Returned solution structure	38
10	Using the Solver	40
10.1	Minimal workflow	40
10.2	Problem definition	40
10.3	Choosing a refinement indicator	41
10.3.1	Undivided Laplacian	41
10.3.2	Relative truncation error	41

10.3.3 Geometry function	41
10.4 Boundary and nullspace cases	41
10.5 The supplied adaptive Poisson demo	42
10.6 The visual walkthrough demo	42
10.7 Practical option guidance	42
10.8 Common interpretation errors	42
11 Implementation Architecture and Extension Points	44
11.1 Hot-path design	44
11.2 Computational cost	44
11.3 Memory lifecycle	45
11.4 Extension points	45
11.4.1 New local operator	45
11.4.2 New indicator	45
11.4.3 New boundary type	45
11.4.4 Different refinement ratio	45
11.5 Implementation invariants checklist	45
11.6 Debugging by symptom	46
11.7 Current scope and limitations	46
A Complete Option Reference	48
B Function-by-Function Reference	49
C Notation and Array Shapes	51
D Suggested Verification Suite	52
D.1 Uniform-grid manufactured convergence	52
D.2 Adaptive manufactured convergence	52
D.3 Mass-correction A/B test	52
D.4 Patch-seam invariance	52
D.5 Pure-Neumann gauge test	52
D.6 Coefficient-compatibility test	53
E Compatibility, Error Semantics, and Proof Boundaries	54
E.1 Which Neumann problems have a gauge?	54
E.2 Compatibility after every refinement	54
E.3 A multidimensional QIF sign counterexample	55

E.4	What the diagnostics establish	55
E.5	Reproducible command-line checks	55
References		57

List of Figures

1.1	Simplified call graph. Geometry-driven tagging builds all requested adaptive levels before the first solve; solution-driven tagging alternates solve, tag, add level, and re-solve.	2
1.2	The three nested loops. Each adaptive stage launches a fixed-hierarchy solve; each solve runs many V-cycles; each V-cycle runs the recursive level transition down to the direct bottom solve and back.	2
3.1	Uniform sub-root grids and adaptive composite levels share one data structure and one recursive V-cycle.	8
3.2	Composite visibility. Coarse cells under a fine footprint are retained for FAS and transfer, but are omitted from composite norms and balances.	8
3.3	Patch lineage is a pair of adjacency lists between adjacent levels, not a tree. A fine patch may have several parents (here fine 2), because clustering does not respect parent-patch boundaries.	9
4.1	A schematic ghost landscape. A single face can be split among landscape classes, while faceInternal only records whether the face is away from the physical boundary.	13
4.2	The QIF/LIF data flow, frame by frame. Colour marks the source of each value: blue=fine data, orange=parent/coarse data, purple=the ghost; a white cell holds no value yet. Frame (a) states the goal on the mesh (fill the purple ghost ring from the blue and orange values we have). The helper QC starts empty (b) and is filled from two independent sources — restriction of fine data into its interior (c) and a gather of parent data into its ring (d). For the ghost fill the kernel reads only that ring (plus the patch’s own fine interior) to compute each ghost (e). No lineage vectors or tree traversal are involved. Frame (f) shows where the dissected patch sits in the overall hierarchy.	15
4.3	The regular QIF coarse–fine ghost stencil. The code rotates and reflects this geometry for all patch faces and corners.	16
6.1	Flux-balance correction at a coarse–fine interface. The coarse equation outside the footprint is altered so its interface term represents the two fine fluxes.	22
6.2	One recursive level transition in the V-cycle. The saved QC state makes the FAS correction a difference of full approximations.	23
6.3	One composite V-cycle on the levels 1–4 hierarchy. Circles are red–black smoothing sweeps (NPre descending, NPost ascending); the diamond is the prefactored direct solve at level 1.	23

8.1	The illustrative first refinement of the dry run. The clustered root box $B_c = [21, 36, 25, 40]$ (sixteen root cells on a side) becomes the level-4 patch $B_f = [41, 72, 49, 80]$ with 32×32 fine cells, shown subdivided two-to-one inside the red footprint.	30
8.2	Chronology of the first solution-driven refinement stage. The first adaptive patch is created only after the root composite solve has converged.	32
8.3	The composite root force during the first adaptive V-cycle. The FAS load replaces the source under the fine footprint (green), the flux-balance correction is scatter-added in the one-cell ring just outside it (orange), and the original physical source is retained everywhere else. . .	34

List of Algorithms

1	Recursive adaptive FAS V-cycle, <code>ebVcycle(H,k)</code>	23
2	Adding one adaptive level, <code>ebAddLevel</code>	27

Chapter 1

Orientation: What the Solver Does

This chapter states the boundary-value problem the solver targets, names the top-level entry points, and introduces the three nested loops and the module map that the rest of the document expands.

1.1 Problem class

The solver addresses the scalar, variable-coefficient elliptic boundary-value problem

$$\mathcal{L}u := -\nabla \cdot (D(x, y) \nabla u(x, y)) + C(x, y)u(x, y) = f(x, y), \quad (x, y) \in \Omega, \quad (1.1)$$

on a rectangular domain

$$\Omega = [a, b] \times [c, d].$$

Each side can independently carry Dirichlet data, $u = g$, or Neumann data,

$$\frac{\partial u}{\partial n} = g,$$

where n is the *outward* unit normal. The implementation requires $D > 0$ at all sampled faces. Nonnegative C is the natural regime for monotonicity and robust point relaxation, although the input routine does not enforce $C \geq 0$.

The spatial discretization is cell-centered and conservative: D is sampled at face midpoints, C and the source are cell-centered, and the diffusion term is written as a difference of face fluxes. Refinement occurs in rectangular patches, with a fixed *refinement ratio* of two between consecutive levels in each direction; the patches themselves may be rectangles of any shape — the clusterer imposes no aspect-ratio constraint, only a minimum width.

Key idea

The numerical solution is not one rectangular array. It is a *composite field*: every physical location is represented by the finest patch covering it, while coarser values under a fine patch are retained as multigrid state and synchronized by restriction.

1.2 Top-level execution path

A typical user creates a problem with `ebProblem`, creates options with `ebOptions`, and calls `ebSolve`. The top-level path is shown in [figure 1.1](#).

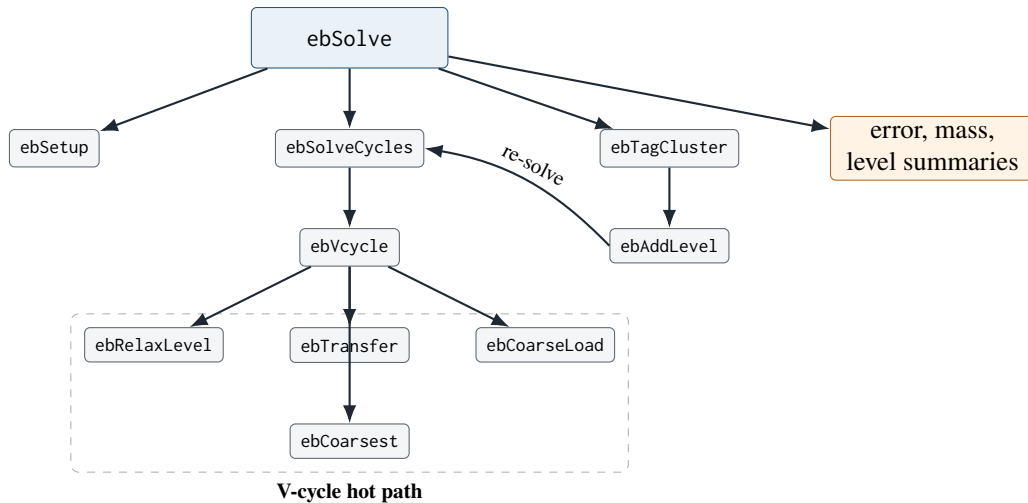


Figure 1.1: Simplified call graph. Geometry-driven tagging builds all requested adaptive levels before the first solve; solution-driven tagging alternates solve, tag, add level, and re-solve.

1.3 The three nested loops

The code is easiest to understand as three nested loops.

1. **Adaptive-stage loop.** Solve the current hierarchy, tag the current finest level, append a new level, and solve again. For a geometric user tag function, hierarchy construction precedes the single composite solve.
2. **V-cycle loop.** On a fixed hierarchy, repeat V-cycles until the composite infinity-norm residual is below $\text{RelTol } \|f\|_\infty$ or MaxVCycles is reached.
3. **Recursive level loop.** A V-cycle pre-smooths, restricts, loads the FAS coarse equation, recursively solves the next coarser level, prolongs a correction, and post-smooths. Level 1 is solved directly.

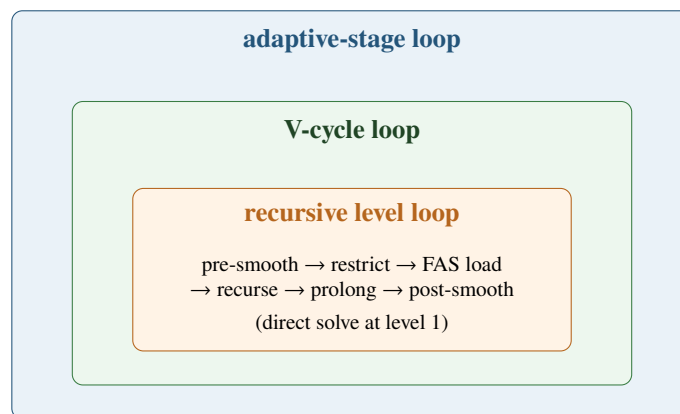


Figure 1.2: The three nested loops. Each adaptive stage launches a fixed-hierarchy solve; each solve runs many V-cycles; each V-cycle runs the recursive level transition down to the direct bottom solve and back.

1.4 Source modules by responsibility

Table 1.1 groups the source files by the role they play in a solve; later chapters take up each group in turn.

Responsibility	Principal files
Public interface	ebProblem, ebOptions, ebSolve
Hierarchy construction	ebSetup, ebMakeLevel, ebAddLevel, ebBuildOps
Discretization kernels	ebKernels, ebBcFold, ebCoarsest
Ghosts and interfaces	ebFillGhosts, ebInterpCf, ebFaceCorrections
Interlevel transfer	ebTransfer, ebGatherWindow, ebCoeffFilldown, ebGatherParentCoeffs, ebParentCoeffs
Multigrid control	ebSolveCycles, ebVcycle, ebRelaxLevel, ebCoarseLoad
Adaptivity	ebTagCluster, ebRte
Diagnostics and output	ebCompResidual, ebCompositeMean, ebErrorNorms, ebMassCheck, ebSample, ebPlot
Examples	demoPoissonAdaptive, demoVisualWalkthrough

Table 1.1: Functional decomposition of the source tree.

Chapter 2

Cell-Centered Discretization

This chapter fixes the cell-centered grid, the face-centered coefficient layout, and the conservative flux-form operator that every level shares. These are the building blocks the multigrid and adaptive machinery later move between scales.

2.1 Level grid and cell centers

On a level with $n_x \times n_y$ logical cells,

$$h_x = \frac{b-a}{n_x}, \quad h_y = \frac{d-c}{n_y}.$$

The cell center (i, j) is

$$x_i = a + \left(i - \frac{1}{2}\right) h_x, \quad y_j = c + \left(j - \frac{1}{2}\right) h_y.$$

A patch is represented by a global index box

$$B = [i_{\min}, i_{\max}, j_{\min}, j_{\max}].$$

If $m_x = i_{\max} - i_{\min} + 1$ and $m_y = j_{\max} - j_{\min} + 1$, the patch solution array has size $(m_x + 2) \times (m_y + 2)$. Local indices $2:m_x + 1$ and $2:m_y + 1$ are interior cells; index 1 and the final index form a one-cell ghost ring.

The local index of global cell (i, j) is

$$r = i - i_{\min} + 2, \quad s = j - j_{\min} + 2.$$

MATLAB's first array dimension is used for the x index and the second for the y index. The code consistently builds coordinate arrays with `ndgrid`, not `meshgrid`.

2.2 Face-centered coefficients

For one patch, the stored arrays are

$$\begin{aligned} \text{DeW}\{p\} &\in \mathbb{R}^{(m_x+1) \times m_y}, & D_{i+1/2,j}^x &= D(x_{i+1/2}, y_j), \\ \text{DnS}\{p\} &\in \mathbb{R}^{m_x \times (m_y+1)}, & D_{i,j+1/2}^y &= D(x_i, y_{j+1/2}), \\ \text{CC}\{p\} &\in \mathbb{R}^{m_x \times m_y}, & C_{ij} &= C(x_i, y_j). \end{aligned}$$

The names `DeW` and `DnS` refer to diffusivities on east–west and north–south faces. The source is stored in `FTrue{p}`, while `F{p}` is a working force that is overwritten during the FAS descent.

2.3 Flux-form operator

Define the discrete face increments

$$\begin{aligned} q_{i+1/2,j}^x &= D_{i+1/2,j}^x (u_{i+1,j} - u_{ij}), \\ q_{i,j+1/2}^y &= D_{i,j+1/2}^y (u_{i,j+1} - u_{ij}). \end{aligned}$$

Then the kernel `op2d` computes

$$(L_h u)_{ij} = -\frac{q_{i+1/2,j}^x - q_{i-1/2,j}^x}{h_x^2} - \frac{q_{i,j+1/2}^y - q_{i,j-1/2}^y}{h_y^2} + C_{ij} u_{ij}. \quad (2.1)$$

Equivalently, if the physical numerical flux is $F_{i+1/2,j}^x = -D_{i+1/2,j}^x (u_{i+1,j} - u_{ij})/h_x$, then the first term is the discrete divergence $(F_{i+1/2,j}^x - F_{i-1/2,j}^x)/h_x$.

```

1 feW = DeW .* (Q(2:mx+2,2:my+1) - Q(1:mx+1,2:my+1));
2 fnS = DnS .* (Q(2:mx+1,2:my+2) - Q(2:mx+1,1:my+1));
3 L = -(feW(2:mx+1,:) - feW(1:mx,:))/(hx*hx) ...
4     -(fnS(:,2:my+1) - fnS(:,1:my))/(hy*hy) ...
5     + CC .* Q(2:mx+1,2:my+1);

```

Listing 2.1: Core operator in `ebKernels`.

For an interior cell, the unfurled diagonal is

$$a_{ij} = \frac{D_{i-1/2,j}^x + D_{i+1/2,j}^x}{h_x^2} + \frac{D_{i,j-1/2}^y + D_{i,j+1/2}^y}{h_y^2} + C_{ij}. \quad (2.2)$$

The reciprocal of this quantity is computed by `invdiag2d`; physical-boundary modifications are then folded into it by `ebBcFold`.

2.4 Source representation

By default, `FTrue{p}` contains midpoint values $f(x_i, y_j)$. With `SourceQuad = n_q > 1`, each cell stores a tensor Gauss–Legendre approximation to the *cell average*

$$\bar{f}_{ij} = \frac{1}{h_x h_y} \int_{K_{ij}} f(x, y) \, dx \, dy.$$

The nodes and weights are generated by a Golub–Welsch eigenvalue calculation. Cell averages are especially useful in all-Neumann problems because the composite sum of visible-cell sources more accurately represents the continuous source integral.

2.5 Operator properties and assumptions

For $D > 0$, $C \geq 0$, and at least one Dirichlet boundary segment or positive reaction, the continuous problem has the usual coercive elliptic structure. This does not imply that the multidimensional QIF composite matrix is a Z -matrix; the explicit stencil in [chapter E](#) demonstrates a positive off-diagonal entry. With all-Neumann data and $C \equiv 0$, constants form a nullspace; the implementation uses a discrete source projection, an augmented mean constraint on the coarsest grid, and an optional composite mean shift after the final solve.

Implementation note

The input routine validates the box domain and boundary types, and level construction checks that sampled face diffusivities are strictly positive. It does not prove coercivity or enforce $C \geq 0$. The singular solver projects its sampled source onto discrete compatibility; callers should inspect `H.compatibility.sourceShift` because this projection also changes incompatible continuous data.

Chapter 3

Hierarchy, Patches, and the Data Model

This chapter describes how the single ratio-two level ladder serves two purposes at once—uniform multigrid support below the root, adaptive patches above it—and catalogs the per-level and per-patch fields, including the precomputed communication descriptors that keep the V-cycle fast.

3.1 Sub-root levels versus adaptive levels

The hierarchy `H.lev{1..nlev}` is a single ratio-two ladder, but its levels play two different roles.

- Levels $1, \dots, \text{rootIdx} - 1$ are full-domain, uniform *sub-root* grids used only as multigrid coarse levels.
- Level `rootIdx` is the user-requested root grid `BaseCells`.
- Levels above `rootIdx` are adaptive patch levels appended by `ebAddLevel`.

Composite residuals, errors, means, mass balances, plots, and user level summaries start at `rootIdx`. Sub-root levels participate in every V-cycle but do not contribute physical degrees of freedom to the composite solution.

For the default `BaseCells=[64 64]` and `CoarsestCells=16`, `ebSetup` constructs 16^2 , 32^2 , and 64^2 full-domain grids, so `rootIdx=3`. A first adaptive level has logical resolution 128^2 , although only its patches are allocated.

3.2 How the uniform ladder is selected

Starting from `BaseCells`, `ebSetup` repeatedly halves both dimensions while all of the following hold:

- (i) both current dimensions are even;
- (ii) the current minimum dimension exceeds `CoarsestCells`;
- (iii) the next minimum dimension would be at least four.

Thus level 1 always has at least four cells in each direction, but it need not equal `CoarsestCells` exactly if divisibility prevents further halving.

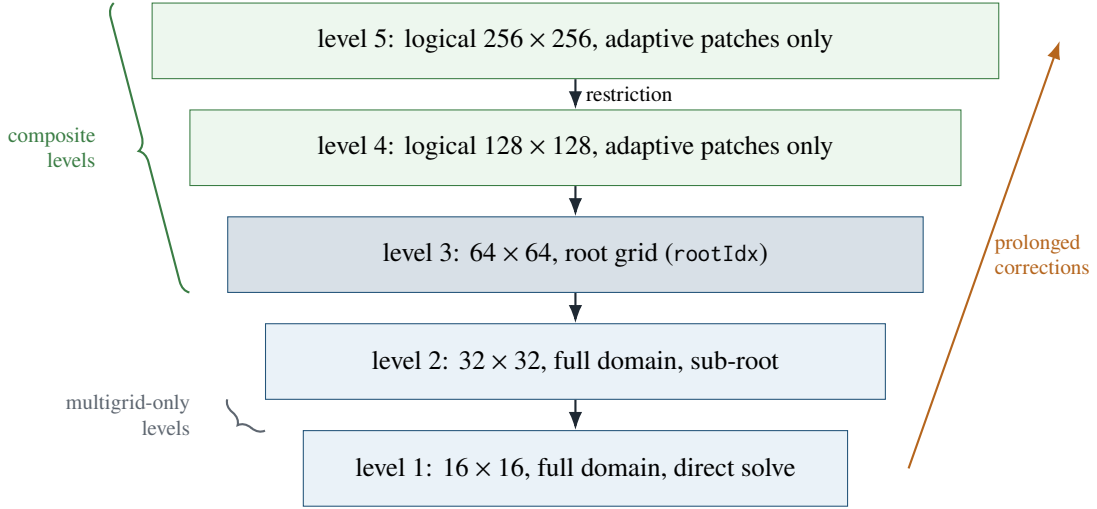


Figure 3.1: Uniform sub-root grids and adaptive composite levels share one data structure and one recursive V-cycle.

3.3 Patches, coverage, and visible cells

Each level stores an array of nonoverlapping rectangular patch boxes. The mask $\text{covered}\{p\}$ marks cells of patch p that are covered by the next finer level. A cell is *visible* if it is not covered:

$$\chi_{k,p}(i, j) = 1 - \text{covered}\{p\}(i, j).$$

The composite domain is the disjoint union of visible cells over levels $k = \text{rootIdx}, \dots, \text{nlev}$.

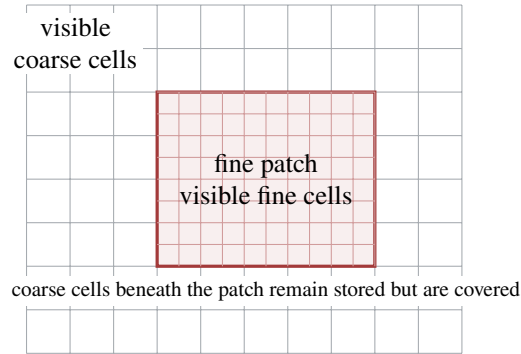


Figure 3.2: Composite visibility. Coarse cells under a fine footprint are retained for FAS and transfer, but are omitted from composite norms and balances.

Definition 3.1 (Composite inner product). *For cell arrays v and w , the code's composite inner product is represented by*

$$(v, w)_{\text{comp}} = \sum_{k=\text{rootIdx}}^{\text{nlev}} \sum_{p=1}^{n_p(k)} \sum_{(i,j) \in \mathcal{V}_{k,p}} h_{x,k} h_{y,k} v_{k,p,ij} w_{k,p,ij}.$$

3.4 Patch lineage and bookkeeping

It is worth being explicit about how the parent–child relationship between levels is stored, because it is *not* a tree. There is no quadtree, octree, or forest-of-octrees, and no spatial index over patches. The hierarchy is a flat cell array of levels $H.\text{lev}\{1..n\text{lev}\}$, and within a level the patches are simply a flat list: $\text{lev}.\text{box}$ is an $n_p \times 4$ array of integer boxes $[i_{\min} \ i_{\max} \ j_{\min} \ j_{\max}]$.

Lineage is recorded as *bidirectional adjacency lists of patch indices between consecutive levels only*. For each fine patch p on level k , $\text{lev}.\text{parents}\{p\}$ is the vector of coarse patch indices on level $k-1$ that its footprint overlaps; symmetrically, $H.\text{lev}\{k-1\}.\text{children}\{q\}$ lists the fine patches over coarse patch q . `ebBuildOps` fills both by a brute-force pairwise rectangle intersection, an $O(n_p^{\text{coarse}} n_p^{\text{fine}})$ scan that is recomputed whenever a level is appended.

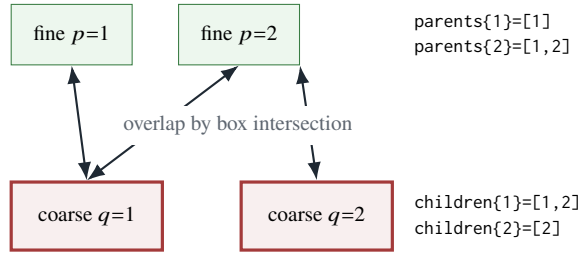


Figure 3.3: Patch lineage is a pair of adjacency lists between adjacent levels, not a tree. A fine patch may have several parents (here fine 2), because clustering does not respect parent-patch boundaries.

Two points follow from this representation. First, the relation is a general adjacency, not a strict tree: because the clusterer does not respect coarse-patch boundaries, a fine patch can intersect more than one coarse patch, so $\text{parents}\{p\}$ may have length greater than one. The proper-nesting and one-level-interface invariants keep the structure forest-like, but it is never traversed as a tree. Second, only adjacent levels are linked; there is no global ancestor chain, and a jump from level k to level $k+2$ is impossible by construction.

These index vectors are bookkeeping. The data motion that actually runs in the V-cycle is driven by the precomputed gather/scatter descriptors of [table 3.3](#), resolved once at build time so the hot path is pure indexed copying: rectangular range descriptors where the region is a box (windows, scatters, coefficient gathers) and grouped linear-index lists where it is irregular (rings, same-level segments, corrections). This flat-array, recompute-on-regrid design is a deliberate fit to serial, block-structured AMR with modest patch counts, and is intentionally simpler than the Morton-ordered tree structures used by large-scale distributed frameworks.

3.5 The hierarchy structure H

The entire solver state lives in a single structure H , whose top-level fields are summarized below; the per-level substructure $H.\text{lev}\{k\}$ is detailed in the next section.

3.6 Per-level and per-patch fields

Each level $H.\text{lev}\{k\}$ carries the following fields. Array shapes are given per patch p , with m_x, m_y the interior patch dimensions; fields written $\{p\}$ are cell arrays indexed by patch.

Field	Meaning
H.prob	Domain, coefficient/source function handles, boundary data, optional exact solution.
H.opts	Parsed solver and adaptivity options.
H.K	Function handles for the operator, inverse diagonal, smoother kernel, restriction, and prolongation.
H.rootIdx	Index of the root AMR level in the complete multigrid ladder.
H.nlev	Total number of levels, including sub-root and adaptive levels.
H.lev{k}	Level structure described below.
H.coarsest	Sparse decomposition, boundary right-hand side, and singularity flag for level 1.
H.stats.stages	One convergence record for each solve stage.

Table 3.1: Top-level hierarchy fields.

Table 3.2: Principal fields of H.lev{k}.

Field	Shape/scope	Meaning
n, h	1×2	Logical grid dimensions and mesh widths.
np, box	scalar, $n_p \times 4$	Number of patches and global index boxes.
Q{p}	$(m_x + 2) \times (m_y + 2)$	Interior solution plus one ghost layer.
FTrue{p}	$m_x \times m_y$	Physical source representation for the level.
F{p}	$m_x \times m_y$	Working FAS force, reset before every V-cycle.
DeW{p}	$(m_x + 1) \times m_y$	Diffusivity on x faces.
DnS{p}	$m_x \times (m_y + 1)$	Diffusivity on y faces.
CC{p}	$m_x \times m_y$	Reaction coefficient.
InvDiagS{p}	$m_x \times m_y$	Inverse diagonal after physical-boundary folding.
par0(p)	scalar	Patch phase that converts local checkerboard indices to global parity.
bc{p}	four records	West, east, south, north codes and sampled boundary data.
faceInternal{p}	four logicals	Whether each patch face is away from the physical boundary.
covered{p}	$m_x \times m_y$ logical	Cells hidden by level $k + 1$.
parents{p}, children{p}	index vectors	Patch overlap bookkeeping across adjacent levels.
land{p}	face vectors	Ghost landscape codes: 1 physical, 2 same-level, 3 coarse-fine.
cornerClass{p}	four integers	The same classification for SW, SE, NW, NE corner ghosts.
sameOps{p}	copy descriptors	Precomputed same-level ghost exchange.
ops{p}	descriptor structure	Parent gathers, scatters, correction targets, and coefficient gathers.
QC{p}	$(m_x/2 + 2) \times (m_y/2 + 2)$	Padded parent-scale window whose interior holds restricted fine data.
cfD{p}	four vectors	Parent diffusivities on the coarse-fine interface, used by mass corrections.

3.7 Precomputed communication descriptors

ebBuildOps constructs owner maps temporarily and converts geometry into indexed copy operations. The V-cycle therefore avoids repeated geometric searches.

Descriptor	Operation
ops.ring	Gather parent Q values into the one-cell ring of QC. Physical parent ghosts may be read when the ring crosses the domain boundary.
ops.winrect	Gather rectangular parent-interior blocks into a complete parent window.
ops.usc	Scatter a restricted fine solution into padded parent Q interiors.
ops.fsc	Scatter FAS coarse loads into unpadded parent F.
ops.corrW/E/S/N	Scatter-add interface corrections to parent cells just outside a fine footprint.
ops.pcc/pdew/pdns	Gather parent C, x-face D, and y-face D over a child footprint.

Table 3.3: Per-patch parent/child operation descriptors.

Implementation invariant 3.1 (Ratio-two alignment). *For every level $k > 1$, a fine patch satisfies*

$$i_{\min}, j_{\min} \text{ odd}, \quad i_{\max}, j_{\max} \text{ even}.$$

Consequently its parent footprint has integer limits $[(i_{\min} + 1)/2, i_{\max}/2, (j_{\min} + 1)/2, j_{\max}/2]$, and both fine patch widths are even.

Implementation invariant 3.2 (Parent-window coverage). *The parent level must cover every cell in a child footprint and its one-cell parent ring. ebBuildOps counts gathered cells and raises a proper-nesting error if the $(m_x/2 + 2) \times (m_y/2 + 2)$ window is incomplete.*

Chapter 4

Ghost Cells and Boundary Closures

This chapter explains how all geometric coupling—physical boundaries, same-level patch seams, and coarse–fine interfaces—is pushed into a one-cell ghost layer so that a single interior stencil applies everywhere, and how the smoother stays exact at boundary cells.

4.1 Why ghost cells carry the coupling

Every patch uses the same five-point operator kernel, including cells next to patch boundaries. All geometric complexity is moved into the ghost layer. Depending on location, a ghost value comes from

1. interpolation from the parent level at a coarse–fine interface;
2. direct copying from a same-level sibling patch;
3. a physical Dirichlet or Neumann closure.

This uniform operator/variable ghost split is one of the code’s main structural simplifications.

4.2 Canonical fill order

ebFillGhosts always applies the three mechanisms in the order

coarse–fine interpolation $\longrightarrow$ same-level exchange $\longrightarrow$ physical boundary closure.

The order is deliberate. A patch face may contain both coarse–fine and same-level segments. The coarse–fine formula is first applied across the whole nonphysical face, then sibling data overwrites the segments that have true same-level neighbors. Finally, physical sides and physical corners take precedence.

4.3 Quadratic Dirichlet closure

Consider a physical boundary at $x = 0$, with a ghost center at $x = -h/2$, the first two interior centers at $x = h/2$ and $x = 3h/2$, and boundary value $g = u(0)$. The quadratic interpolant through g, u_1, u_2 gives

$$u_g = \frac{8}{3}g - 2u_1 + \frac{1}{3}u_2. \quad (4.1)$$

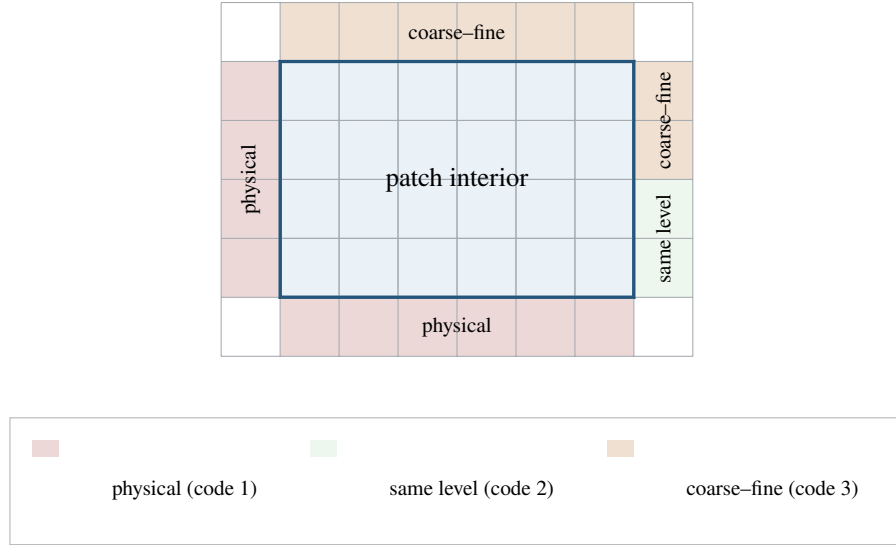


Figure 4.1: A schematic ghost landscape. A single face can be split among landscape classes, while `faceInternal` only records whether the face is away from the physical boundary.

The same formula, with the appropriate orientation, is used on all four sides. Physical corner ghosts are then filled by bilinear-consistent extrapolation, for example

$$u_{g,SW} = u_{1,gS} + u_{gW,1} - u_{1,1}.$$

When [equation \(4.1\)](#) is substituted into the boundary-cell equation, the boundary face contributes

$$\frac{D_b}{h^2} \left(3u_1 - \frac{1}{3}u_2 - \frac{8}{3}g \right). \quad (4.2)$$

The coarsest sparse matrix includes this expression directly.

4.4 Neumann closure and sign convention

The user supplies the outward normal derivative $g = \partial u / \partial n$. On every side the centered ghost relation is

$$u_g = u_1 + hg. \quad (4.3)$$

For example, at the west boundary $n = -e_x$ and $(u_g - u_1)/h \approx -u_x = \partial u / \partial n$; at the east boundary the same algebraic formula approximates u_x .

Substitution of [equation \(4.3\)](#) removes the boundary-face unknown coupling and contributes $D_b g / h$ to the right-hand side of the ghost-eliminated equation.

4.5 Exact boundary treatment in red–black relaxation

A naive smoother would fill the complete ghost formula and then use the ordinary diagonal. That update is not exact Gauss–Seidel for the ghost-eliminated boundary equation because the ghost itself contains the

unknown boundary-cell value. The code instead splits the ghost formula into a part independent of u_1 and a part proportional to u_1 .

For a Dirichlet side,

$$u_g = \underbrace{\frac{8}{3}g + \frac{1}{3}u_2}_{\text{written into the ghost}} \underbrace{-2u_1}_{\text{folded into the diagonal}},$$

so `ebBcFold` adds $2D_b/h^2$ to the ordinary diagonal. Since the ordinary diagonal already includes D_b/h^2 , the total boundary-face diagonal becomes $3D_b/h^2$, matching [equation \(4.2\)](#).

For a Neumann side,

$$u_g = \underbrace{hg}_{\text{written into the ghost}} + \underbrace{u_1}_{\text{folded out}},$$

so `ebBcFold` subtracts D_b/h^2 and cancels the ordinary boundary-face diagonal.

After every colored pass, `ebFillGhosts` restores the complete ghost values needed by the operator and residual kernels.

Key idea

`InvDiagS` is not simply the reciprocal of the five-point diagonal near a physical boundary. It is the reciprocal of the *ghost-eliminated* diagonal. This is why point relaxation at boundary cells is algebraically exact for frozen neighbors.

4.6 Coarse–fine interpolation

For a fine patch at level k , `QC{p}` is a parent-scale array with one ghost ring. Its ring contains parent values immediately outside the fine footprint. Coarse–fine interpolation uses these parent values together with fine interior values to fill fine ghosts.

It helps to see the data flow as six frames, [figure 4.2](#). Throughout, colour marks the *source* of each value: **blue** for fine data, **orange** for parent (coarse) data, and **purple** for the ghost values we must compute; a *white* cell means “no value stored yet.” Colour marks the *role* of a value, not the grid level: in particular the purple ghost cells lie on the *same* fine level as the blue interior — they are the patch’s own ghost layer, never a finer patch. **Watch the scale:** `QC` is a *parent-scale* array, so panels (b)–(d) draw it with cells twice the fine size, whereas (a), (e), and (f) are at the fine-mesh scale. Frame (a) states the goal: fill the fine patch’s one-cell *ghost ring* (purple) from the values we already have — the fine interior (blue) and the surrounding parent cells (orange). The buffer `QC` is the workspace; it begins empty (b), gets its interior written by restriction (c), and its ring written by the gather ops. `ring` (d). For the ghost fill only the *ring* of `QC` matters: `ebInterpCf` blends each ring value u_C (coarse, one per $2h$) with two of the patch’s *own* fine interior values u_{F1}, u_{F2} (fine, spacing h). That mismatch of scales is exactly the change in grid density you see between (d) and (e). The kernel does this for every cell of the ring (e), without walking a tree or the parents/children lists. (The restricted interior of (c) is *not* used for the ghost fill; it feeds the coarse load of [chapter 6](#).) Frame (f) shows the patch and its ghost ring on the full parent domain — the ghost ring belonging to the *same* level k as the patch — with the coarser multigrid levels beneath it given in [figure 3.1](#).

Remark 4.1 (Why restrict into `QC` if ghosts use fine u_h ?). *Ghost fill reads only the ring of `QC` together with the patch’s own fine interior `Q`; it never uses the restricted interior `Ru_h`. That interior is written anyway*

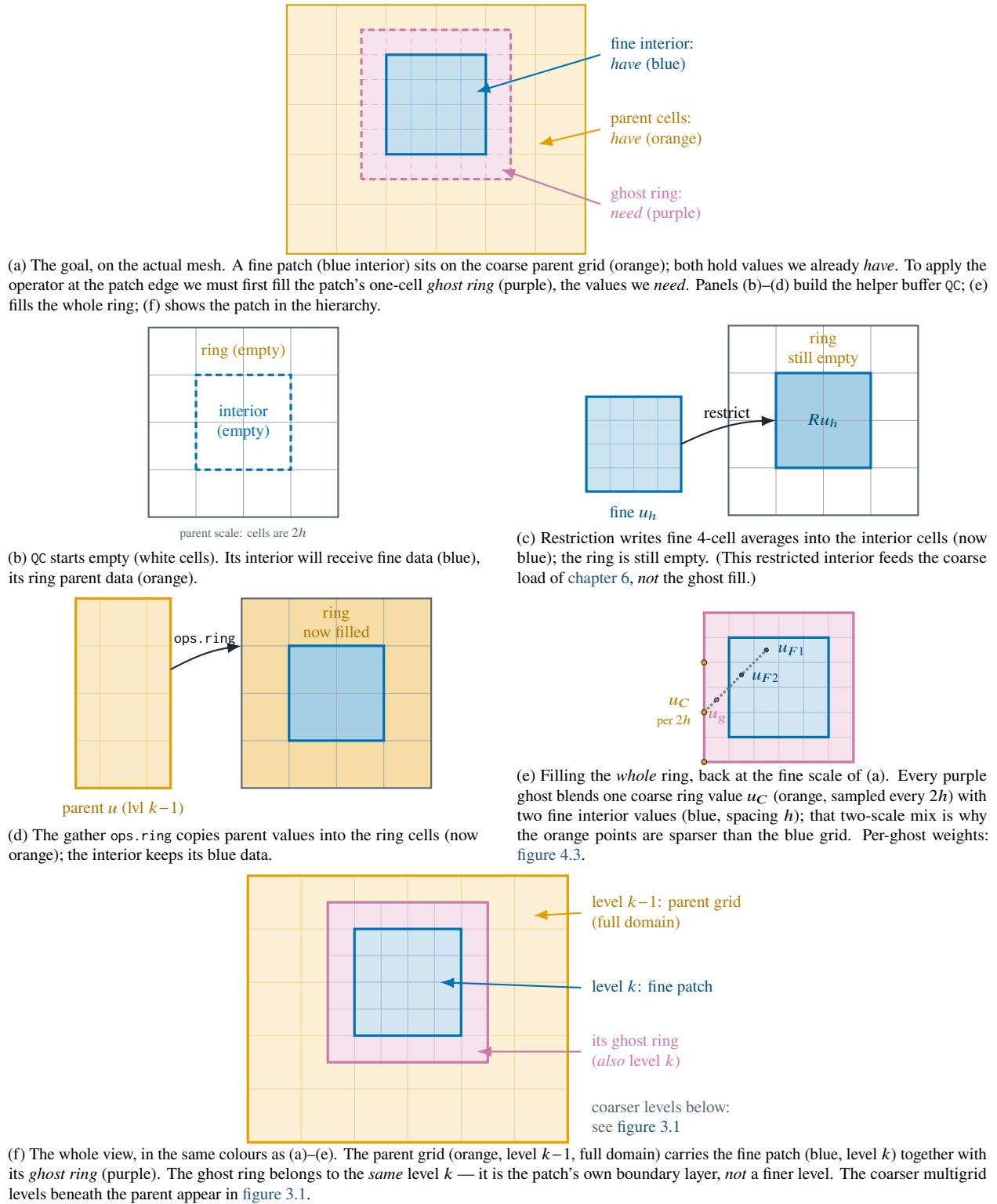


Figure 4.2: The QIF/LIF data flow, frame by frame. Colour marks the source of each value: **blue**=fine data, **orange**=parent/coarse data, **purple**=the ghost; a white cell holds no value yet. Frame (a) states the goal on the mesh (fill the purple ghost ring from the blue and orange values we have). The helper QC starts empty (b) and is filled from two independent sources — restriction of fine data into its interior (c) and a gather of parent data into its ring (d). For the ghost fill the kernel reads only that ring (plus the patch's own fine interior) to compute each ghost (e). No lineage vectors or tree traversal are involved. Frame (f) shows where the dissected patch sits in the overall hierarchy.

because QC is shared parent-scale storage for the whole FAS descent: it supplies $L_{2h}(Ru_h)$ in the coarse load (chapter 6), the restricted values at coarse–fine interfaces in the face-balance corrections, and the pre-coarse-solve reference in the correction prolongation (equation (5.3)). Restriction also overwrites the parent Q under the footprint.

4.6.1 Quadratic interpolation (QIF)

For a regular face ghost, the QIF stencil lies on a 45° line through a diagonally adjacent coarse center u_C and two fine interior values u_{F1}, u_{F2} :

$$u_g = \frac{8}{15}u_C + \frac{2}{3}u_{F1} - \frac{1}{5}u_{F2}. \quad (4.4)$$

The geometry and weights are illustrated in figure 4.3.

At the two end cells of a face, the diagonal stencil would leave the patch. The code first constructs a coarse value at the required tangential station with

$$u_C^* = \frac{5}{32}u_{C,-1} + \frac{15}{16}u_{C,0} - \frac{3}{32}u_{C,1}, \quad (4.5)$$

and then uses the normal quadratic formula. A coarse–fine corner ghost uses the diagonal version of equation (4.4).

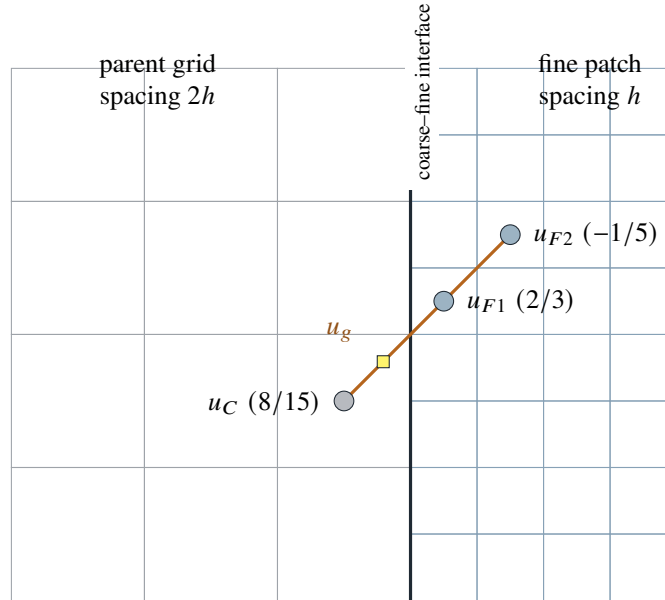


Figure 4.3: The regular QIF coarse–fine ghost stencil. The code rotates and reflects this geometry for all patch faces and corners.

4.6.2 Linear interpolation (LIF)

The LIF option uses

$$u_g = \frac{2}{3}u_C + \frac{2}{3}u_{F,\text{same}} - \frac{1}{3}u_{F,\text{partner}}, \quad (4.6)$$

with alternating partner rows or columns. It has no special end exception. The corner formula reduces to

$$u_g = \frac{2}{3}u_C + \frac{1}{3}u_{F,\text{diag}}.$$

4.7 Same-level exchange and global consistency

ebBuildOps records, for every sibling segment, the source patch, source linear indices, and destination ghost indices. ebFillGhosts then performs indexed copies from sibling interiors. Because the copy occurs after coarse–fine interpolation, a mixed face is handled without branching inside the interpolation kernel.

The smoother uses *global* checkerboard parity. For patch box lower corner $(i_{\min}, j_{\min})$, the phase

$$\text{par}\theta = \text{mod}(i_{\min} + j_{\min}, 2)$$

ensures that all patches update the same global color together. Ghost exchange after each color makes a colored pass equivalent to one half-sweep on the union of same-level patches.

Implementation invariant 4.1 (Current ghosts). *Any call that evaluates $L_h u$, computes a residual, restricts an interface state for a mass correction, or smooths a level assumes the relevant ghost values are current. The recursive cycle and solve-stage driver enforce this ordering explicitly.*

Chapter 5

Interlevel Transfer and Coefficient Compatibility

This chapter covers the data motion between levels—restriction, the padded parent window, bilinear prolongation—and the coefficient fill-down that makes the coarse operator compatible with the fine one. The distinction between the saved QC buffer and a freshly gathered parent window W is central to the FAS correction.

5.1 Restriction

The restriction kernel is the conservative four-cell average

$$(R_h^{2h}v)_{IJ} = \frac{1}{4} \sum_{\alpha=0}^1 \sum_{\beta=0}^1 v_{2I-1+\alpha, 2J-1+\beta}. \quad (5.1)$$

In `ebTransfer(..., 'restrict')`, the result is written to two places:

- (a) the interior of the child patch's QC array;
- (b) the parent Q cells under the child footprint, through `ops.usc`.

The second write keeps covered coarse data synchronized with the fine approximation during the FAS descent and during the post-cycle fill-down.

5.2 The padded parent window

For a child patch of size $m_x \times m_y$, its parent footprint has size $c_x = m_x/2$ by $c_y = m_y/2$. Two arrays with this parent geometry appear repeatedly:

- QC: a persistent/transient $(c_x + 2) \times (c_y + 2)$ buffer. Its interior is Ru_h after restriction; its ring is gathered from the parent.
- W : a freshly materialized parent window returned by `ebGatherWindow`. It contains the *current* parent solution over both the footprint and its ring.

The distinction is crucial during correction. QC stores the parent-scale approximation before the recursive coarse solve, whereas W stores the parent solution after that solve.

5.3 Bilinear prolongation

The prolongation kernel maps a padded coarse array W to $2c_x \times 2c_y$ fine values. For the southwest child of a coarse cell C , the weights are

$$(PW)_{2I-1,2J-1} = \frac{9}{16}C + \frac{3}{16}W_- + \frac{3}{16}S_- + \frac{1}{16}D_{--}, \quad (5.2)$$

with reflected variants for the other three children. The one-cell coarse ring supplies the neighbor values needed at the fine-patch boundary.

During a V-cycle, prolongation acts on a *coarse correction*:

$$u_h \leftarrow u_h + P_h^{2h} (W - QC). \quad (5.3)$$

During level creation, the same kernel acts directly on the parent solution window to initialize the new fine patch.

5.4 Coefficient fill-down

Each newly created level initially samples the analytic coefficient functions at its own faces and centers. The hierarchy is then made interlevel-compatible by `ebCoeffFilldown`. Wherever a coarse level is covered by a fine patch,

$$D_{I+1/2,J}^{x,2h} = \frac{1}{2} \left(D_{2I+1/2,2J-1}^{x,h} + D_{2I+1/2,2J}^{x,h} \right), \quad (5.4)$$

$$D_{I,J+1/2}^{y,2h} = \frac{1}{2} \left(D_{2I-1,2J+1/2}^{y,h} + D_{2I,2J+1/2}^{y,h} \right), \quad (5.5)$$

$$C_{IJ}^{2h} = \frac{1}{4} \sum_{\alpha,\beta=0}^1 C_{2I-1+\alpha,2J-1+\beta}^h. \quad (5.6)$$

Thus a coarse face coefficient is the mean of the two collinear fine faces, while a coarse cell coefficient is the mean of four fine cells. Fill-down proceeds from the finest level to level 1, then every relaxation diagonal is recomputed.

Key idea

The coarse operator used by FAS is not an independently sampled rediscretization under a fine footprint. It is made compatible with the fine operator by coefficient averaging. This same parent operator is used in the coarse load, interface correction, and direct bottom solve.

5.5 Transient parent coefficients

A child patch may overlap multiple parent patches. `ebGatherParentCoeffs` precomputes rectangular descriptors for parent C , D^x , and D^y . During each coarse load, `ebParentCoeffs` materializes the footprint arrays transiently, applies the parent operator, and discards them.

Only the four interface-face diffusivity vectors $cfD.W/E/S/N$ are stored persistently. This reduces permanent metadata from patch-area storage to patch-perimeter storage while keeping the coarse-load code vectorized.

5.6 Transfer modes in ebTransfer

Mode	Direction	Effect
restrict	fine $\rightarrow$ parent	Fill QC interior with Ru_h and overwrite parent Q under the footprint.
ring	parent $\rightarrow$ QC	Gather the parent ring around each child footprint.
correct	parent $\rightarrow$ fine	Gather current parent window W , form $W - QC$, prolong, and add to fine interiors.
filldown	finest $\rightarrow$ level 1	Cascade restriction and parent ghost fills through the entire hierarchy.

Table 5.1: Interlevel data-motion modes.

5.7 Preconditions on QC

The meaning of QC depends on where the code is in the cycle:

- coarse–fine ghost interpolation needs only a valid ring;
- the FAS coarse load needs both the restricted interior and the gathered ring;
- face corrections need the restricted first interior row/column and the gathered ring;
- correction needs the unchanged pre-coarse-solve QC as a reference state.

The returned solved hierarchy releases QC arrays to save memory. Internal routines that are called later must rebuild the required state in the same order as the V-cycle: restrict first when an interior is needed, then gather the ring.

Chapter 6

Adaptive FAS Multigrid and Conservative Interfaces

This chapter assembles the components into the recursive AFAS V-cycle. It motivates the full-approximation formulation, derives the flux-balance interface correction that makes the composite scheme conservative, and describes the smoother, the direct bottom solve, and the per-cycle force lifecycle.

6.1 Why FAS is used for a linear equation

The PDE in this project is linear, so a correction-scheme multigrid method would be possible. The implementation nevertheless uses full approximation storage. On each level it solves for the full approximation u_k , and the coarse force is chosen so that restricting an exact fine solution produces a consistent coarse equation. This organization matches the adaptive, conservative formulation and extends naturally to nonlinear problems.

6.2 The FAS coarse equation

Let u_h and f_h be the current fine approximation and working force. On coarse cells covered by the fine patch, the code forms

$$f_{2h}^{\text{FAS}} = R_h^{2h}(f_h - L_h u_h) + L_{2h}(R_h^{2h} u_h). \quad (6.1)$$

The first term is the restricted fine residual. The second applies the compatible parent operator to the restricted full approximation stored in the padded QC array. `ops.fsc` scatters the result into the appropriate parent patches. Parent cells outside the footprint retain their physical `FTrue` source, except for the interface correction described next.

6.3 Why ghost interpolation alone is not conservative

Fine ghost interpolation gives the fine operator a well-defined stencil at an interface, but it does not by itself make the coarse flux through one coarse face equal the aggregate flux through the two corresponding fine faces. If the coarse equation outside the fine footprint continues to use its original coarse face flux, summing equations over the composite grid leaves a non-canceling interface term.

The code therefore modifies the *coarse force* in cells immediately outside the fine footprint. The fine equations remain unchanged; the coarse equation is made to see the fine flux.

6.4 Flux-balance correction

Consider a west face of a fine patch. One coarse interface face corresponds to two fine faces. Write $u_{\text{in},1}^f$ and $u_{\text{in},2}^f$ for the fine interior values, $u_{g,1}^f$ and $u_{g,2}^f$ for their interpolated ghosts, u_{out}^c for the parent cell outside the footprint, and $[Ru^f]_{\text{in}}$ for the restricted fine value inside. The code adds

$$C_W = \frac{\sum_{\alpha=1}^2 D_{\alpha}^f (u_{\text{in},\alpha}^f - u_{g,\alpha}^f) + D^c (u_{\text{out}}^c - [Ru^f]_{\text{in}})}{h_{x,c}^2} \quad (6.2)$$

to the coarse force in the outside cell. East, south, and north formulas are rotations of [equation \(6.2\)](#); south/north use $h_{y,c}^2$.

The sign in the last term removes the coarse operator's old interface contribution while the sum of fine terms inserts the restricted aggregate fine contribution. `ebFaceCorrections` evaluates the values, and the precomputed `ops.corrW/E/S/N` descriptors scatter-add them to parent force arrays.

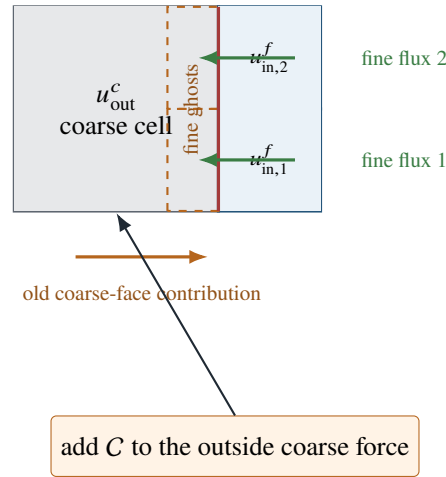


Figure 6.1: Flux-balance correction at a coarse–fine interface. The coarse equation outside the footprint is altered so its interface term represents the two fine fluxes.

6.5 Recursive V-cycle

The recursive routine is almost a literal transcription of the AFAS algorithm.

Unrolling that recursion across the four-level hierarchy of [chapter 8](#) gives the familiar V schedule of [figure 6.3](#): smoothing sweeps on the way down to the bottom, one direct solve at level 1, and smoothing sweeps on the way back up.

Algorithm 1 Recursive adaptive FAS V-cycle, ebVcycle(H, k)

-
- 1: **if** $k = 1$ **then**
 - 2: solve the prefactored level-1 system and fill its ghosts
 - 3: **return**
 - 4: **end if**
 - 5: perform NPre red–black Gauss–Seidel sweeps on level k
 - 6: restrict u_k into QC and the parent solution under the footprint
 - 7: fill parent ghosts
 - 8: gather the parent ring into QC
 - 9: build the FAS parent force using [equation \(6.1\)](#)
 - 10: add flux-balance corrections using [equation \(6.2\)](#), if enabled
 - 11: recursively call the V-cycle on level $k - 1$
 - 12: gather the new parent window W and set $u_k \leftarrow u_k + P(W - QC)$
 - 13: fill level- k ghosts
 - 14: perform NPost red–black Gauss–Seidel sweeps on level k
-

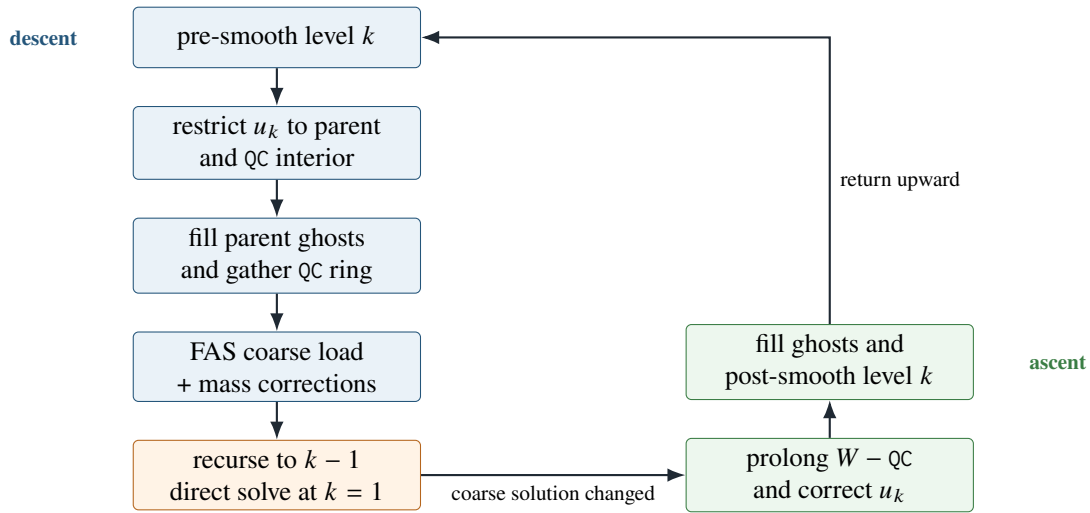


Figure 6.2: One recursive level transition in the V-cycle. The saved QC state makes the FAS correction a difference of full approximations.

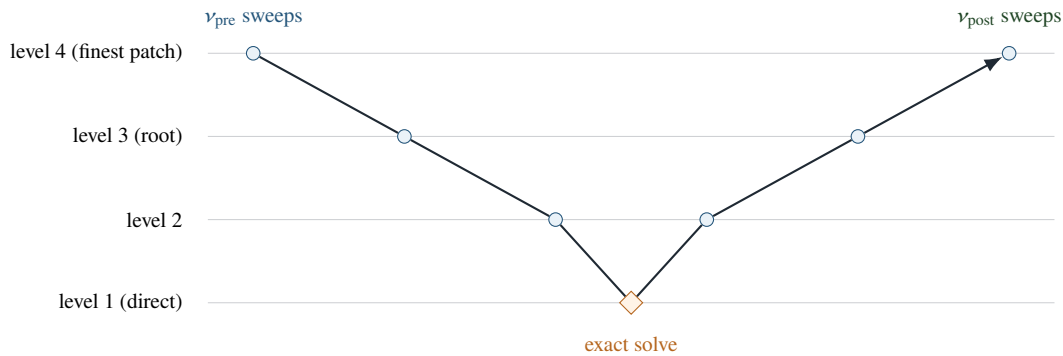


Figure 6.3: One composite V-cycle on the levels 1–4 hierarchy. Circles are red–black smoothing sweeps (NPre descending, NPost ascending); the diamond is the prefactored direct solve at level 1.

6.6 Red–black Gauss–Seidel smoother

For a fixed color, the kernel forms the exact point solution

$$u_{ij}^* = \frac{1}{a_{ij}^S} \left(f_{ij} + \frac{D_{i+1/2,j}^x}{h_x^2} u_{i+1,j} + \frac{D_{i-1/2,j}^x}{h_x^2} u_{i-1,j} + \frac{D_{i,j+1/2}^y}{h_y^2} u_{i,j+1} + \frac{D_{i,j-1/2}^y}{h_y^2} u_{i,j-1} \right), \quad (6.3)$$

where a_{ij}^S is the boundary-folded diagonal. The stored value is

$$u_{ij} \leftarrow \omega u_{ij}^* + (1 - \omega) u_{ij}.$$

For $\Omega=1$, this is red–black Gauss–Seidel on ordinary interior and physical-boundary rows. Coarse–fine ghost values are held fixed during a color pass and refreshed afterwards; this is not literal Gauss–Seidel on the fully eliminated QIF composite matrix, whose wider stencil can couple cells of the same color.

6.7 The level-1 direct solve

`ebCoarsest(H, 'assemble')` constructs the sparse ghost-eliminated matrix once for the current hierarchy. Internal faces add symmetric two-cell contributions. Physical boundaries are folded using [equation \(4.2\)](#) and [equation \(4.3\)](#). MATLAB’s decomposition object is stored in `H.coarsest.dec`.

When all physical boundaries are Neumann and every sampled reaction coefficient is exactly zero, the code marks the problem singular and factors

$$\begin{bmatrix} A & e \\ e^T & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}, \quad e = (1, \dots, 1)^T. \quad (6.4)$$

The final row pins the unweighted mean of the uniform coarsest solution. After the complete adaptive solve, `MeanZero='auto'` additionally subtracts the *composite* visible-cell mean from every level.

The direct factorization is rebuilt after every new adaptive level because coefficient fill-down can change the level-1 operator.

6.8 One V-cycle as a force lifecycle

At the start of every V-cycle, `ebSolveCycles` resets every `F{p}` to `FTrue{p}`. During descent:

- covered parent cells are overwritten by FAS loads;
- adjacent visible parent cells receive correction additions;
- all other parent cells keep their true source.

The next V-cycle begins from a clean source state. This reset is essential because corrections and FAS loads are state-dependent and must not accumulate across cycles.

Chapter 7

Adaptive Refinement

This chapter describes how new levels are decided and built: the refinement indicators, tag buffering and proper-nesting constraints, the signature-based box clustering, and the mechanics of appending a level to the hierarchy.

7.1 Two refinement workflows

ebSolve supports two stage orderings.

7.1.1 Solution-driven indicators

For Indicator='ulap' or 'rte', the hierarchy evolves as

solve current hierarchy $\longrightarrow$ tag $\longrightarrow$ cluster $\longrightarrow$ add level $\longrightarrow$ re-solve.

This is repeated until no cells are tagged or the requested number of AMR levels is reached.

7.1.2 Geometry-driven indicator

For Indicator='fun', the user function depends only on cell-center coordinates (and optionally the current AMR level). The code therefore builds all possible levels first and performs one composite solve afterward. This avoids intermediate solves whose only purpose would be to discover geometry already known to the user.

7.2 Undivided-Laplacian indicator

For each fine-level patch, the code computes the unscaled second differences

$$\begin{aligned}\delta_x^2 u_{ij} &= u_{i+1,j} - 2u_{ij} + u_{i-1,j}, \\ \delta_y^2 u_{ij} &= u_{i,j+1} - 2u_{ij} + u_{i,j-1},\end{aligned}$$

and tags where

$$\eta_{ij}^{\text{ulap}} = \sqrt{(\delta_x^2 u_{ij})^2 + (\delta_y^2 u_{ij})^2} > C_k. \quad (7.1)$$

Because the differences are not divided by h^2 , a smooth solution gives $\eta = O(h^2)$. A level-dependent threshold is therefore often scaled by approximately 1/4 under each factor-two refinement.

7.3 Relative truncation-error indicator

The alternative 'rte' mode estimates

$$\tau_{2h} = L_{2h}(Ru_h) - R(L_h u_h). \quad (7.2)$$

The code constructs a padded Ru_h field by restricting interior values and averaging fine ghost pairs onto a parent-scale ring. It also coarsens D^x, D^y, C using the same averages as coefficient fill-down. The absolute value $|\tau_{2h}|$ is then duplicated onto each corresponding 2×2 block of fine cells for tagging.

This is a local indicator rather than a global error estimate. Its purpose is to locate regions where the current discretization is not self-consistent across scales.

7.4 User tag function

With Indicator='fun', TagFunction can be

$$@(\mathbf{X}, \mathbf{Y}) \dots \quad \text{or} \quad @(\mathbf{X}, \mathbf{Y}, L) \dots,$$

where $L = 1$ denotes the root AMR level and refining level L creates level $L + 1$. The return value is converted to a logical array at cell centers.

7.5 Buffering and proper nesting

All patch-local tags are assembled in a logical mask on the current finest logical grid. The mask is dilated by a square Chebyshev neighborhood of radius TagBuffer. Buffering keeps features away from the new patch boundary and reduces rapid regridding around moving or sharply localized structures.

Next the code forms the coverage mask of the current finest patch union and erodes it by the full 3×3 neighborhood. A cell is allowed for refinement if it and all in-domain eight-neighbors are covered by the current level. Points outside the physical domain count as covered, so a child patch may touch a physical boundary.

Implementation invariant 7.1 (One-level interfaces). *Only the current finest level is tagged, and child footprints are clipped to a one-cell erosion of its coverage. Therefore every new coarse-fine interface separates adjacent levels; the hierarchy never jumps directly from level k to level $k + 2$.*

7.6 Signature-based box clustering

The clustering routine is a compact Berger–Rigoutsos-style signature splitter.

1. Tighten the current box around its tagged cells.
2. Accept it if its fill ratio meets or exceeds MinFillRatio, or if both widths are already no larger than MinBoxWidth.
3. Otherwise form one-dimensional signatures by summing tags along each coordinate.

4. Prefer a split at a zero-signature hole, choosing the admissible hole nearest the center.
5. If no hole exists, split at the strongest jump in the discrete second difference of a signature.
6. If no signature split is admissible, bisect the longest dimension when possible.
7. Recursively process the two children.

After clustering, boxes are bisected until they lie completely in the proper-nesting region. The code then tries to grow each box to `MinBoxWidth` without leaving the allowed region or overlapping another box. Any remaining one-cell sliver is merged with a nearby box when the union is admissible and compact; otherwise it is dropped with a warning.

Implementation note

`MinBoxWidth` is a design target, not an unconditional postcondition in a narrow allowed region. The implementation guarantees that surviving boxes are not one-cell slivers, but a box can remain between two cells and `MinBoxWidth`–1 cells wide if geometric constraints prevent further growth.

7.7 Appending a new level

Suppose a clustered parent-space box is

$$B_c = [I_\ell, I_r, J_b, J_t].$$

`ebAddLevel` maps it to the aligned fine box

$$B_f = [2I_\ell - 1, 2I_r, 2J_b - 1, 2J_t]. \quad (7.3)$$

The new logical level size is twice the parent size. The sequence is then:

Algorithm 2 Adding one adaptive level, `ebAddLevel`

- 1: map parent boxes to aligned fine boxes using [equation \(7.3\)](#)
 - 2: create new patch arrays and sample D, C, f , boundary data, and diagonals
 - 3: fill coefficients downward through the complete hierarchy
 - 4: build new parent/child, same-level, gather/scatter, and correction descriptors
 - 5: rebuild parent coefficient metadata and persistent interface diffusivities
 - 6: reassemble the level-1 direct factorization
 - 7: fill parent ghosts
 - 8: **for** each new fine patch **do**
 - 9: gather the full padded parent window W
 - 10: set the fine interior to PW
 - 11: **end for**
 - 12: fill the new level's coarse–fine, sibling, and physical ghosts
-

7.8 The adaptive-stage stopping rule

MaxLevels counts only the root and adaptive levels:

$$\text{AMR level count} = \text{nlev} - \text{rootIdx} + 1.$$

It does not count the uniform sub-root ladder. Refinement also stops early when ebTagCluster returns no boxes.

Chapter 8

A Dry Run: From the Bottom Solve to the First Adaptive Level

This chapter follows one concrete solve, array by array, from hierarchy construction and the bottom factorization through the first root V-cycle, the first tagging and refinement, and the first composite V-cycle on the enlarged hierarchy.

8.1 Purpose of the tour

This chapter follows actual data movement rather than presenting another abstract algorithm. The running configuration uses

$$\text{BaseCells}=[64 \ 64], \quad \text{CoarsestCells}=16, \quad \text{MaxLevels} \geq 2,$$

with a solution-driven indicator. The initial hierarchy is

$$\underbrace{16^2}_{k=1} \longrightarrow \underbrace{32^2}_{k=2} \longrightarrow \underbrace{64^2}_{k=3=\text{rootIdx}}.$$

All three initial levels are full-domain single patches.

To make the first refinement concrete, suppose the root-level clusterer later returns

$$B_c = [21, 36, 25, 40],$$

a 16×16 parent-space box. The child patch is

$$B_f = [41, 72, 49, 80]$$

on the logical 128^2 level, has 32×32 interior cells, a 34×34 padded Q, and an 18×18 parent-scale QC window. [Figure 8.1](#) shows this box in place on the root grid.

8.2 Stage 0: hierarchy construction and bottom factorization

ebSetup first creates the three full-domain levels. It samples coefficients and sources, fills coefficients down from 64^2 to 32^2 and 16^2 , builds transfer descriptors between levels 2/1 and 3/2, initializes any user guess, and fills all ghosts.

64 × 64 root (level 3), drawn in 4-cell blocks

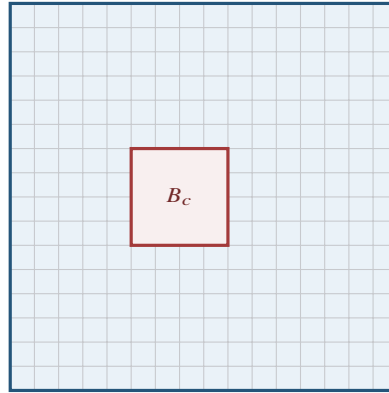


Figure 8.1: The illustrative first refinement of the dry run. The clustered root box $B_c = [21, 36, 25, 40]$ (sixteen root cells on a side) becomes the level-4 patch $B_f = [41, 72, 49, 80]$ with 32×32 fine cells, shown subdivided two-to-one inside the red footprint.

It then calls

```
ebCoarsest(H, 'assemble').
```

At this moment no PDE solve has occurred. The code has only assembled and factored the 16^2 ghost-eliminated matrix, together with its boundary right-hand side. This factorization is the bottom engine that every later V-cycle will use.

Code path

```
ebSolve → ebSetup → ebMakeLevel → ebCoeffFilldown → ebBuildOps → ebFillGhosts → ebCoarsest('assemble').
```

8.3 Stage 1, first V-cycle: descend from the root

The first root solve calls `ebSolveCycles`, which measures the root composite residual and then invokes

```
ebVcycle(H, 3).
```

Because there is no adaptive level yet, only level 3 is composite-visible. Levels 1 and 2 are purely multigrid support.

8.3.1 Level 3 to level 2

1. `ebRelaxLevel(H, 3, NPre)` smooths the 64^2 root field. Since the level has one full-domain patch, only physical ghosts are involved.
2. `ebTransfer(..., 'restrict')` averages every 2×2 block into level 2. The entire 32^2 parent is covered, so its Q interior becomes Ru_3 .
3. Level-2 ghosts are filled. QC ring data for the level-3 patch is gathered from level 2.
4. `ebCoarseLoad(H, 3)` overwrites the full level-2 force with $R(f_3 - L_3u_3) + L_2(Ru_3)$.

There are no coarse cells outside the footprint, because the fine level covers the whole domain. Therefore the interface correction lists are empty on this uniform transition.

8.3.2 Level 2 to level 1

The same sequence repeats from 32^2 to 16^2 : pre-smooth, restrict, fill ghosts, gather the ring, and load the complete level-1 FAS force.

8.4 The bottom event: direct level-1 solve

The recursion reaches `ebVcycle(H,1)`, which immediately calls

$$\text{ebCoarsest}(H, \text{'solve'}).$$

The current level-1 working force is not merely the physical 16^2 source. It is the FAS force produced by the two descent steps. The direct solve applies the prefactored matrix to this current right-hand side, writes the 16^2 solution into the interior of $Q\{1\}$, and refills physical ghosts.

This is the conceptual bottom of the whole solver: all low-frequency error that survived smoothing on the root and intermediate grid has been encoded into the coarsest FAS equation and solved exactly, subject to the mean constraint if necessary.

8.5 Ascend from level 1 to the root

Returning to level 2, the code gathers the now-updated level-1 window W_1 , subtracts the saved pre-solve QC_2 , and bilinearly prolongs the difference:

$$u_2 \leftarrow u_2 + P(W_1 - QC_2).$$

It fills level-2 ghosts and post-smooths. The same operation then corrects level 3 from level 2:

$$u_3 \leftarrow u_3 + P(W_2 - QC_3).$$

After root post-smoothing, the V-cycle returns.

`ebSolveCycles` now performs a hierarchy fill-down. Covered coarse data are reset to restrictions of finer data, and parent ghosts are refreshed. On this full-domain ladder, the entire 32^2 and 16^2 interiors are synchronized with the levels above. The composite residual is measured only on the 64^2 root.

This root-stage V-cycle repeats until convergence.

8.6 The first tagging pass

Once the root stage converges, `ebTagCluster` examines level 3. For the illustrative box B_c :

1. the indicator exceeds its threshold in a localized set;
2. tags are buffered by `TagBuffer`;

3. the root coverage is eroded for proper nesting (the full root makes every in-domain root cell admissible);
4. signature splitting produces $B_c = [21, 36, 25, 40]$.

The box is expressed in *root-level cell indices*, not in the future fine-level indices.

8.7 Creating level 4

`ebAddLevel` transforms B_c into $B_f = [41, 72, 49, 80]$ and calls `ebMakeLevel` with logical size 128^2 .

The new patch allocates:

$$\begin{aligned}
 Q\{p\} &: 34 \times 34, \\
 F\text{True}\{p\}, F\{p\}, CC\{p\} &: 32 \times 32, \\
 DeW\{p\} &: 33 \times 32, \\
 DnS\{p\} &: 32 \times 33, \\
 QC\{p\} &: 18 \times 18.
 \end{aligned}$$

Next, coefficient fill-down replaces root coefficients under the 16×16 parent footprint by averages of the new fine coefficients, then propagates compatible values through the sub-root ladder. Transfer and interface descriptors are built. The root patch's covered mask becomes true on cells 21:36, 25:40. The bottom matrix is reassembled because these coefficient changes can reach level 1.

8.8 Initializing the new fine patch

Before any composite solve, the code fills root ghosts and gathers the 18×18 root window

$$W_3 = \text{root values on the } 16 \times 16 \text{ footprint plus one parent ring.}$$

The new 32×32 fine interior is set to PW_3 . This is an initial guess, not a conservative synchronization. Finally `ebFillGhosts(H, 4)` performs QIF or LIF interpolation on coarse–fine segments, same-level exchange if there are multiple new patches, and physical closures if the patch touches the domain boundary.

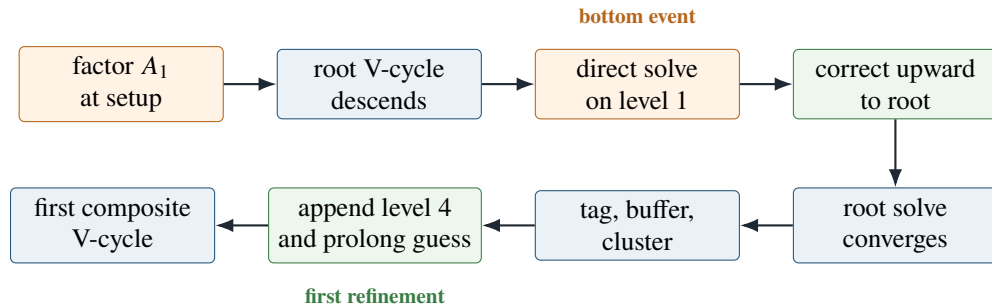


Figure 8.2: Chronology of the first solution-driven refinement stage. The first adaptive patch is created only after the root composite solve has converged.

8.9 The first composite V-cycle on levels 1–4

The next solve stage calls `ebVcycle(H, 4)`. This time the first descent step is genuinely adaptive.

8.9.1 Pre-smooth the fine patch

Level 4 is smoothed with coarse–fine ghosts supplied from level 3. If the new level has several touching patches, shared segments are overwritten by sibling interiors after the interpolation pass.

8.9.2 Restrict the fine patch

Restriction produces a 16×16 array Ru_4 . It is written into the interior of the 18×18 QC_4 buffer and into the root cells 21:36, 25:40. Root ghosts are then refilled, and the one-cell root ring around the footprint is gathered into QC_4 .

At this point:

- QC_4 interior is the pre-coarse-solve Ru_4 ;
- QC_4 ring is the current root solution outside the footprint;
- root Q under the footprint has also been overwritten by Ru_4 .

8.9.3 Load the root FAS equation

Inside root cells 21:36, 25:40, `ebCoarseLoad` writes

$$R(F_4 - L_4 u_4) + L_3(Ru_4).$$

For every coarse–fine face around the footprint, `ebFaceCorrections` computes the sum of two fine face contributions, removes the old root interface contribution, and adds the resulting correction to the root cell just outside the footprint.

Root cells far from the footprint still have `FTrue3`. Thus the root equation is a composite mixture:

physical source away from the patch, FAS load beneath the patch, and conservative correction beside the interface.

Figure 8.3 maps these three regions on the root grid.

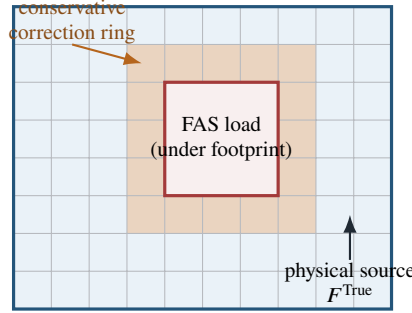


Figure 8.3: The composite root force during the first adaptive V-cycle. The FAS load replaces the source under the fine footprint (green), the flux-balance correction is scatter-added in the one-cell ring just outside it (orange), and the original physical source is retained everywhere else.

8.9.4 Recurse through root and sub-root levels

The cycle now calls `ebVcycle(H, 3)`. From here the recursion resembles the earlier uniform solve, except that the root force already contains the adaptive coarse load and interface terms. This information is restricted through levels 2 and 1, solved at the bottom, and propagated back to the root.

8.9.5 Correct level 4

After the recursive root solve, `ebTransfer(..., 'correct')` gathers the *new* root window W_3 . The saved QC_4 still contains the pre-root-solve parent-scale state. Therefore

$$E_3 = W_3 - QC_4$$

is the root correction over the footprint and its ring. Bilinear prolongation updates the fine interior:

$$u_4 \leftarrow u_4 + PE_3.$$

The fine ghosts are refilled and level 4 is post-smoothed.

8.9.6 Synchronize and measure

After the V-cycle, fill-down restricts level 4 back into the covered root cells and continues through the sub-root ladder. The composite residual is measured on

- visible root cells outside B_c ;
- all visible level-4 cells inside B_f .

Interface corrections are included in the root residual beside the patch. The solve stage repeats V-cycles until the composite residual satisfies the stopping criterion.

8.10 State table for the adaptive descent/ascent

Table 8.1: State of the principal arrays during the level-4/level-3 transition.

Moment	Array	Meaning
Before descent	level-4 Q	Current fine full approximation with current ghosts.
After restriction	level-4 QC interior	Ru_4 saved as the pre-coarse-solve reference.
After restriction	level-3 Q footprint	Also overwritten with Ru_4 .
After ring gather	level-4 QC ring	Current root values just outside the footprint.
After coarse load	level-3 F footprint	$R(F_4 - L_4u_4) + L_3(Ru_4)$.
After correction add	level-3 F outside ring	Physical source plus conservative interface correction.
After recursive solve	level-3 Q	New root approximation; covered and visible values may both change.
During ascent	W_3	New root window gathered after the recursive solve.
During ascent	$W_3 - QC_4$	Coarse full-approximation correction to prolong.
After post-smooth	level-4 Q	Updated fine approximation.
After fill-down	level-3 Q footprint	Resynchronized to Ru_4 for a consistent stored hierarchy.

8.11 What changes at a second refinement

After the two-level composite stage converges, the indicator is evaluated on level 4 only. The proper-nesting mask is now the one-cell erosion of the level-4 patch union. A level-5 patch can therefore be created only where a complete level-4 neighborhood exists, except at physical boundaries. The same add-level and composite-cycle logic repeats without introducing a new algorithmic case.

Chapter 9

Convergence Control and Diagnostics

This chapter defines the composite residual that drives the stopping rule, the solve-stage driver, the error and mass-conservation diagnostics, and the sampling/plotting utilities and returned solution structure.

9.1 Composite residual

For a visible cell away from an interface, the residual is simply

$$r_{k,p} = F_{k,p}^{\text{True}} - L_k u_{k,p}.$$

On a coarse visible cell adjacent to a finer patch, the same flux-balance correction used in the FAS coarse force is added to the residual force. This makes `ebCompResidual` measure the residual of the conservative composite fixed-point problem, not the residual of independent per-level discretizations.

The reported norms are

$$\|r\|_{\infty} = \max_{k,p,(i,j) \in \mathcal{V}_{k,p}} |r_{k,p,ij}|, \quad (9.1)$$

$$\|r\|_2 = \left(\sum_{k,p} \sum_{(i,j) \in \mathcal{V}_{k,p}} h_{x,k} h_{y,k} r_{k,p,ij}^2 \right)^{1/2}. \quad (9.2)$$

The relative value used for stopping is

$$\text{res.rel} = \frac{\|r\|_{\infty}}{\max(\|F^{\text{True}}\|_{\infty, \text{visible}}, \text{realmin})}. \quad (9.3)$$

This is a source-relative tolerance, not a reduction relative to the initial residual.

9.2 Solve-stage driver

`ebSolveCycles` performs the following sequence on the current hierarchy:

1. fill ghosts on every level;

2. measure the initial composite residual;
3. while `it < MaxVCycles` and `res.rel > RelTol`:
 - (a) reset all working forces to true sources;
 - (b) run one V-cycle from the finest level;
 - (c) fill down the solution hierarchy;
 - (d) measure the composite residual;
4. store the stage history and convergence statistics.

For residual history $r_0, \dots, r_m$, the average geometric convergence factor is

$$\rho_{\text{avg}} = \left(\frac{r_m}{\max(r_0, \text{realmin})} \right)^{1/m}. \quad (9.4)$$

9.3 Error norms

If an exact solution is supplied, `ebErrorNorms` evaluates it at every composite cell center and reports

$$\|e\|_{\infty} = \max_{\text{visible}} |u_h - u|, \quad (9.5)$$

$$\|e\|_2 = \left(\sum_{\text{visible}} h_x h_y (u_h - u)^2 \right)^{1/2}. \quad (9.6)$$

The structure also contains a per-AMR-level table

$$[L, \|e_L\|_{\infty}, \|e_L\|_2, N_{\text{visible},L}].$$

For an all-Neumann problem, the solution is defined only up to a constant when $C \equiv 0$. Exactly in that singular case, the error routine removes the composite mean of the error for every `SourceQuad`. If reaction is present, the absolute error is reported. The removed constant is returned as `err.meanShift`; the solution itself is unchanged by this diagnostic. Source quadrature changes the right-hand side, not the operator's nullspace.

9.4 Composite mean and final gauge

`ebCompositeMean` computes

$$\bar{u}_{\text{comp}} = \frac{\sum_{\text{visible}} h_x h_y u_h}{\sum_{\text{visible}} h_x h_y}. \quad (9.7)$$

With `MeanZero='auto'`, `ebSolve` subtracts this mean from every patch on every level only when the coarsest operator was classified as singular. A numeric/logical true value forces the shift; false suppresses it.

9.5 Discrete mass-conservation check

Integrating [equation \(1.1\)](#) over the domain gives

$$\int_{\Omega} (Cu - f) \, dx \, dy = \int_{\partial\Omega} D \frac{\partial u}{\partial n} \, ds.$$

The discrete counterpart evaluated by `ebMassCheck` is

$$\sum_{\text{visible}} h_x h_y (C_{ij} u_{ij} - F_{ij}^{\text{True}}) = \sum_{\partial\Omega} D \frac{u_g - u_l}{h_n} h_t. \quad (9.8)$$

Here h_n is the normal spacing and h_t the tangential face length. Covered physical-boundary cells are excluded so that each boundary segment is counted at the finest visible resolution.

The returned fields are `lhs`, `flux`, `defect=lhs-flux`, and a normalized `rel`. The check is logically independent of the residual norm:

- a small residual says the algebraic composite equations have been solved;
- a small mass defect says the interface construction telescopes correctly when those equations are summed.

With `MassCorrection=false`, the code still solves a defined adaptive discretization, but the interface flux cancellation measured by [equation \(9.8\)](#) is not enforced.

9.6 Sampling and plotting

`ebSample` accepts query arrays (x_q, y_q) and searches levels from finest to root. The first patch covering a point supplies a bilinear interpolation from its padded Q array. Because physical and coarse–fine ghost closures are already present, interpolation remains defined up to patch edges.

`ebPlot` has two modes:

- 'mesh' draws patch rectangles colored by AMR level;
- 'solution' samples the composite field on a uniform query grid and draws a single surface, avoiding overlapping coarse and fine surfaces.

9.7 Returned solution structure

Implementation note

In `sol.levels(L).n`, n is the full logical resolution of AMR level L , even when only a small fraction is allocated. The `cells` field is the sum of allocated patch interiors, not the number of visible cells; stage statistics separately report `visibleCells`.

Field	Content
<code>sol.H</code>	Complete hierarchy, including sub-root levels and the final patch state.
<code>sol.stats</code>	Per-stage V-cycle counts, residual histories, final relative residuals, average factors, visible cells, and allocated AMR cells.
<code>sol.err</code>	Composite error norms when an exact solution is available; otherwise empty.
<code>sol.mass</code>	Discrete conservation diagnostic.
<code>sol.levels</code>	Root/adaptive summaries: logical size, mesh width, patch count, allocated patch cells, and boxes.

Table 9.1: Fields returned by `ebSolve`.

Chapter 10

Using the Solver

This chapter is the practical guide: a minimal end-to-end workflow, how to define problems and boundary data, how to choose a refinement indicator, the supplied demos, and a table of common interpretation errors.

10.1 Minimal workflow

The public workflow is compact:

```
1 uex = @(x,y) sin(pi*x).*sin(pi*y);
2 f   = @(x,y) (2*pi^2 + 1).*uex(x,y); % -Delta u + u
3
4 prob = ebProblem([0 1 0 1], ...
5                 1, ... % D
6                 1, ... % C
7                 f, ...
8                 {'dirichlet', 0}, ...
9                 uex);
10
11 opts = ebOptions('BaseCells', [64 64], ...
12                 'MaxLevels', 3, ...
13                 'Indicator', 'ulap', ...
14                 'TagThreshold', @(L) 1e-3/4^(L-1), ...
15                 'RelTol', 1e-10);
16
17 sol = ebSolve(prob, opts);
18 fprintf('Linf error = %.3e\n', sol.err.linf);
19 fprintf('mass defect = %.3e\n', sol.mass.rel);
```

Listing 10.1: Minimal manufactured-solution workflow.

10.2 Problem definition

The signature is

```
prob = ebProblem(domain,D,C,f,bc,exact).
```

domain is [a b c d]. The coefficient/source arguments may be scalars or vectorized function handles @(X,Y) returning arrays of the same size as X and Y. A single boundary specification {'dirichlet',g} or {'neumann',g} applies to all sides. Mixed data use a structure with fields west, east, south, and north.

```

1 bc.west = {'dirichlet', @(x,y) 0*x};
2 bc.east = {'neumann',   @(x,y) cos(y)};
3 bc.south = {'dirichlet', 1};
4 bc.north = {'neumann',   0};
5 prob = ebProblem([a b c d], D, C, f, bc);

```

Listing 10.2: Mixed physical boundary data.

Remember that Neumann g is the *outward* derivative on each side. The solver converts scalar data to arrays automatically.

10.3 Choosing a refinement indicator

10.3.1 Undivided Laplacian

Use 'ulap' when sharp curvature is the desired refinement signal. Since the indicator scales like h^2 for a smooth function, a threshold function such as

$$C_L = C_1/4^{L-1}$$

is often more meaningful than a constant threshold.

10.3.2 Relative truncation error

Use 'rte' when operator self-consistency across adjacent scales is a better signal than raw curvature, especially for variable coefficients. Its magnitude has operator units, so threshold tuning depends on the PDE scale.

10.3.3 Geometry function

Use 'fun' for prescribed regions, interfaces known analytically, or parameter studies where the mesh should not depend on the current numerical solution. The hierarchy is built before the solve.

10.4 Boundary and nullspace cases

For pure Neumann data with $C \equiv 0$:

- the continuous compatibility relation should be satisfied;
- the coarsest solve uses the augmented system [equation \(6.4\)](#);
- MeanZero='auto' shifts the final composite mean to zero;
- SourceQuad>1 can reduce source-integral drift on mixed-resolution meshes.

For a reaction problem with $C > 0$, an all-Neumann boundary condition is nonsingular and the automatic mean shift is not applied.

10.5 The supplied adaptive Poisson demo

`demoPoissonAdaptive` constructs a manufactured solution formed from three narrow Gaussian bumps, sets $D = C = 1$, uses exact Dirichlet data, and refines with the undivided-Laplacian indicator. It demonstrates the intended high-level pattern:

1. define u_{exact} and derive f ;
2. create the problem and adaptive options;
3. solve and print error/mass diagnostics;
4. export mesh and composite-solution figures.

10.6 The visual walkthrough demo

`demoVisualWalkthrough` is an executable narrative. It manually exposes setup, root solve, indicator, tags, clustered boxes, level addition, QIF stencil anatomy, composite re-solve, error map, and final statistics. In desktop MATLAB it behaves like a slideshow; it also exports individual frames, a GIF, and an MP4.

10.7 Practical option guidance

Goal	Suggested starting choice
Robust default interface	<code>GhostInterp='qif', MassCorrection=true</code> .
Cheaper interface experiment	<code>GhostInterp='lif'</code> while retaining mass correction.
Classical V-cycle balance	<code>NPre=3, NPost=3, Omega=1</code> .
Sharp localized features	<code>Indicator='ulap'</code> with level-scaled threshold and a 2–3 cell buffer.
Coefficient-driven difficulty	Try <code>Indicator='rte'</code> .
Known refinement geometry	<code>Indicator='fun'</code> to avoid intermediate solves.
Pure Neumann accuracy study	Keep <code>MeanZero='auto'</code> and consider <code>SourceQuad=2</code> or higher.
Conservation study	Compare <code>MassCorrection=true/false</code> and inspect <code>sol.mass.rel</code> .

Table 10.1: Reasonable initial choices for common goals. Thresholds remain problem-dependent.

10.8 Common interpretation errors

1. **“Level 1” is not the root.** It is the bottom full-domain grid. AMR level 1 is `H.rootIdx`.
2. `MaxLevels` **does not count sub-root levels.** It counts root plus adaptive levels.
3. **A converged residual is not a discretization error estimate.** Tagging and manufactured-solution norms address spatial resolution.

4. **Fine ghost interpolation and conservation correction are separate.** Disabling `MassCorrection` does not disable QIF/LIF.
5. **Covered coarse cells are not additional composite unknowns.** They are synchronization and multigrid state.
6. **The `ulap` indicator is unscaled.** A constant threshold changes meaning with h .

Chapter 11

Implementation Architecture and Extension Points

This chapter steps back to the engineering view: how the hot path is kept cheap, the cost and memory model, where to extend the solver safely, and a checklist of invariants and symptom-driven debugging hints.

11.1 Hot-path design

The source is organized so that geometry-heavy work happens when a level is built, while a V-cycle consists mostly of array kernels and indexed copies.

- Owner maps are temporary in `ebBuildOps`; per-cell geometric searches are not repeated during smoothing or transfer.
- Parent window interiors use rectangular slice descriptors; irregular rings use grouped linear-index gathers.
- Same-level ghost exchange is pre-resolved by source patch.
- Interface correction destinations are pre-resolved and only values are recomputed.
- Parent coefficient footprint arrays are materialized only during a coarse load.
- The working source initially shares MATLAB's copy-on-write buffer with the true source.

11.2 Computational cost

Let N_{alloc} be the total number of allocated patch cells over all levels participating in a V-cycle. Operator application, smoothing, restriction, prolongation, and coarse loading are $O(N_{\text{alloc}})$ per cycle. Ghost and interface metadata scale primarily with patch perimeter, although same-level copies and full-window gathers still touch the data they move.

The level-1 sparse factorization is a setup cost. It is reused over all V-cycles on a fixed hierarchy and rebuilt after every `ebAddLevel` (coefficient fill-down can change the level-1 operator). The direct solve cost is small when `CoarsestCells` is chosen appropriately.

11.3 Memory lifecycle

Large persistent arrays are `Q`, `FTrue`, `F`, face coefficients, reaction coefficients, and smoother diagonals. The principal temporary or releasable objects are:

- `QC`, one parent-scale window per non-bottom patch, released at the end of `ebSolve`;
- transient parent coefficient arrays in `ebParentCoeffs`;
- patch-local residual arrays in `ebCompResidual`, created, normed, and discarded one patch at a time;
- correction chunks, which are perimeter-sized.

11.4 Extension points

11.4.1 New local operator

The operator, inverse diagonal, point update, restriction, and prolongation are centralized in `ebKernels`. A new scalar elliptic stencil can often be introduced there, provided all boundary folding, coarsest assembly, coefficient compatibility, and interface correction formulas are updated consistently.

11.4.2 New indicator

A new indicator belongs in the switch in `ebTagCluster`. It should return one value per current-finest-level cell and must respect filled ghost data if it uses a stencil. The existing clustering and proper-nesting machinery can remain unchanged.

11.4.3 New boundary type

Adding Robin or periodic boundaries is not a local change. It requires coordinated updates to problem parsing, level boundary metadata, ghost filling, smoother diagonal folding, coarsest assembly, proper-nesting/ring handling at the domain edge, and mass diagnostics.

11.4.4 Different refinement ratio

The ratio two is embedded in box alignment, restriction, prolongation weights, `QC` dimensions, coefficient averaging, face pairing in the mass correction, RTE upsampling, and clustering-to-child mapping. Generalizing the ratio requires a systematic rewrite of those components.

11.5 Implementation invariants checklist

1. Every fine patch is parent-aligned and has even interior dimensions.
2. Fine patches are nonoverlapping within a level.
3. A child footprint plus its one-parent-cell ring is covered by the parent level.

4. Coarse/fine interfaces separate adjacent levels only.
5. Face diffusivities are strictly positive.
6. Ghosts are current before any operator or smoother call.
7. QC interior is restricted fine data before a coarse load or face correction.
8. Working forces are reset before every V-cycle.
9. Global checkerboard parity is used across all patches of a level.
10. Composite sums include visible cells from root to finest and exclude sub-root grids.

11.6 Debugging by symptom

Table 11.1: Useful first checks when behavior is unexpected.

Symptom	First checks
Patch alignment error	Verify boxes are supplied in parent index space to <code>ebAddLevel</code> ; do not pass already-refined indices.
Parent window not fully covered	Check proper nesting, level jumps, and any manually constructed patches.
Wrong Neumann sign	Confirm that the supplied data are the outward derivative, not a fixed Cartesian derivative.
Divergence near physical boundaries	Check boundary data vectorization and consistency of <code>ebBcFold/coarsest</code> formulas after any stencil modification.
Divergence across patch seams	Inspect same-level ownership, global parity, and ghost refresh after each color.
Small residual but large mass defect	Confirm <code>MassCorrection=true</code> ; inspect interface coefficient fill-down and face correction signs.
Unexpectedly many patches	Increase <code>MinFillRatio</code> , <code>MinBoxWidth</code> , or adjust the tag threshold/buffer.
No refinement	Inspect indicator magnitude and remember that <code>ulap</code> is $O(h^2)$ for smooth fields.
Pure-Neumann offset	Check compatibility, <code>MeanZero</code> , and source cell averaging.
Incorrect post-solve internal residual call	Reconstruct restriction buffers in cycle order before using routines that require QC interiors.

11.7 Current scope and limitations

The supplied implementation is deliberately focused:

- two spatial dimensions;
- rectangular Cartesian domain;

- scalar positive diffusion $D(x, y)$, not a tensor;
- scalar reaction $C(x, y)$;
- Dirichlet and outward-Neumann data, no Robin or periodic boundary type;
- fixed factor-two refinement;
- one ghost layer;
- serial MATLAB patch loops;
- rectangular Berger–Rigoutsos-style patches;
- one direct full-domain bottom solve.

These constraints are also the reason the implementation can keep the numerical kernels compact and the interface geometry explicit.

Appendix A

Complete Option Reference

Table A.1: Options parsed by ebOptions.

Name	Default	Meaning
BaseCells	[64 64]	Root-grid logical dimensions. A scalar is duplicated. Values are rounded; further validation occurs during hierarchy construction.
MaxLevels	1	Number of AMR levels including the root; sub-root grids are excluded.
CoarsestCells	16	Target stopping size for uniform sub-root coarsening, subject to even divisibility and a minimum next size of four.
GhostInterp	'qif'	'qif' for quadratic or 'lif' for linear coarse-fine ghost interpolation.
MassCorrection	true	Add flux-balance corrections to coarse forces and composite residuals.
NPre	3	Red-black Gauss-Seidel sweeps before coarse transfer.
NPost	3	Red-black Gauss-Seidel sweeps after prolonged correction.
Omega	1.0	Relaxation factor in the colored point update.
MaxVCycles	60	Maximum V-cycles per fixed-hierarchy solve stage.
RelTol	1e-9	Stop when the composite residual satisfies equation (9.3) .
Indicator	'ulap'	'ulap', 'rte', or 'fun'.
TagFunction	[]	Logical function of (X,Y) or (X,Y,L) used by 'fun'.
TagThreshold	1e-3	Scalar or function of AMR level for 'ulap' and 'rte'.
TagBuffer	2	Square tag dilation radius in current-finest-level cells.
MinBoxWidth	4	Desired minimum cluster width in parent cells.
MinFillRatio	0.70	Tag density at which a candidate cluster box is accepted.
Verbose	1	0 silent, 1 stage summaries, 2 per-cycle residual output.
InitialGuess	[]	Function @(X,Y) applied to every initial uniform level. New adaptive levels are initialized by parent prolongation.
MeanZero	'auto'	Final composite mean shift. Auto activates only for the singular all-Neumann, zero-reaction case.
SourceQuad	1	Midpoint source for 1 or less; tensor Gauss-Legendre cell average for an integer greater than 1.

Appendix B

Function-by-Function Reference

Table B.1: The solver source files (src/) and principal demos. Test drivers in tests/ and the remaining example scripts in examples/ are not listed.

Function/file	Responsibility
ebProblem	Normalize the domain, coefficient/source handles, side boundary specifications, and optional exact solution.
ebOptions	Parse hierarchy, interface, multigrid, adaptivity, and diagnostic options.
ebSolve	Top-level adaptive workflow; return hierarchy, statistics, errors, conservation check, and level summaries.
ebSetup	Build the initial uniform sub-root/root ladder, transfers, ghosts, and bottom factorization.
ebMakeLevel	Allocate patch arrays; sample coefficients, source, and boundary data; construct base diagonals and metadata.
ebAddLevel	Append one adaptive level, rebuild static machinery, and prolong the parent solution as its initial guess.
ebBuildOps	Precompute ownership classifications, same-level copies, parent windows, restriction/force scatters, correction targets, and coverage masks.
ebCoeffFilldown	Average fine face/cell coefficients onto covered parent regions and recompute relaxation diagonals.
ebGatherParentCoeffs	Build parent coefficient gather descriptors and persistent coarse interface diffusivity vectors.
ebParentCoeffs	Materialize parent D^x , D^y , C arrays over one child footprint for a coarse load.
ebKernels	Provide the flux operator, inverse diagonal, colored smoother kernel, four-cell restriction, and bilinear prolongation.
ebBcFold	Compute physical-boundary diagonal increments for exact ghost-eliminated point relaxation.
ebFillGhosts	Fill coarse–fine, same-level, and physical ghosts in canonical order.
ebInterpCf	Evaluate QIF or LIF face/corner coarse–fine ghost formulas.
ebRelaxLevel	Perform global-parity red–black Gauss–Seidel sweeps and refresh ghosts after each color.
ebTransfer	Restrict, gather rings, prolong corrections, or cascade fill-down.
ebGatherWindow	Assemble a complete padded parent solution window from one or more parent patches.
ebCoarseLoad	Form the FAS parent force on covered cells and add conservative interface corrections.
ebFaceCorrections	Compute per-face flux-balance correction values for one child patch.

Function/file	Responsibility
ebVcycle	Recursive AFAS V-cycle; dispatch the bottom direct solve.
ebCoarsest	Assemble/factor and solve the full-domain level-1 ghost-eliminated system; augment singular Neumann cases.
ebSolveCycles	Repeat fixed-hierarchy V-cycles, synchronize levels, measure residuals, and record stage statistics.
ebTagCluster	Build tag masks, buffer/clip for nesting, and cluster tags into parent-space boxes.
ebRte	Compute and upsample the relative truncation-error indicator.
ebCompResidual	Evaluate conservative composite residual norms on visible cells.
ebCompositeMean	Compute a visible-cell, area-weighted composite mean.
ebErrorNorms	Compute composite L^2 and L^∞ errors and per-level counts.
ebMassCheck	Evaluate the discrete global conservation identity.
ebSample	Bilinearly sample the finest available patch at arbitrary points.
ebPlot	Export adaptive mesh or sampled composite-solution figures.
demoPoissonAdaptive	Concise adaptive manufactured-solution example with mesh/solution output.
demoVisualWalkthrough	Animated pedagogical tour with indicators, tags, patches, interface stencil, errors, and statistics.

Appendix C

Notation and Array Shapes

Mathematical object	Code representation	Notes
Patch box B	<code>lev.box(p, :)</code>	<code>[ilo ihi jlo jhi]</code> in global level indices.
Full approximation u_h	<code>lev.Q{p}</code>	Padded by one ghost layer.
Physical source f_h	<code>lev.FTrue{p}</code>	Midpoint or cell-average representation.
Working FAS force	<code>lev.F{p}</code>	Reset each cycle, overwritten on descent.
x -face diffusion	<code>lev.DeW{p}</code>	$(m_x + 1) \times m_y$.
y -face diffusion	<code>lev.DnS{p}</code>	$m_x \times (m_y + 1)$.
Reaction C_h	<code>lev.CC{p}</code>	Cell centered.
Restricted child state	<code>lev.QC{p}</code>	Parent-scale interior plus ring.
Current parent window	local variable <code>W</code>	Gathered after coarse solve for correction.
Restriction R	<code>H.K.restrict2</code>	Four-cell average.
Prolongation P	<code>H.K.prolong</code>	Cell-centered bilinear interpolation.
Visibility χ	<code>lev.covered{p}</code>	Used in all composite sums.
Interface correction C	<code>ebFaceCorrections</code>	Added to parent force/residual outside a child footprint.

Table C.1: Translation between mathematical notation and code arrays.

Appendix D

Suggested Verification Suite

A robust extension or refactor should be checked with several independent tests.

D.1 Uniform-grid manufactured convergence

Set `MaxLevels=1`, use smooth variable D , nonnegative C , and an exact solution. Refine `BaseCells` and verify second-order spatial convergence of the error norms. This isolates the finite-difference and physical-boundary closures from AMR effects.

D.2 Adaptive manufactured convergence

Use localized smooth features, a level-scaled indicator, and several AMR depths. Check that the error decreases with finest spacing while allocated cells remain substantially below a uniform finest grid.

D.3 Mass-correction A/B test

Run the same adaptive problem with `MassCorrection=true` and `false`. The conservative run should exhibit a much smaller `sol.mass.rel`; the comparison directly exercises interface correction assembly and destination scatters.

D.4 Patch-seam invariance

Represent the same uniform level once as one patch and once as several touching patches, holding all other data fixed. Results should agree to solver tolerance. This exercises global red-black parity and same-level ghost exchange.

D.5 Pure-Neumann gauge test

Choose a compatible manufactured problem with all-Neumann data and $C = 0$. Verify residual convergence, near-zero final composite mean under `MeanZero='auto'`, and error convergence modulo the additive constant.

D.6 Coefficient-compatibility test

Use strongly varying but positive $D(x, y)$. After level creation, independently average fine face coefficients and compare them with covered parent values. This protects `ebCoeffFilldown`, parent coefficient gathers, and interface diffusivity vectors.

Appendix E

Compatibility, Error Semantics, and Proof Boundaries

E.1 Which Neumann problems have a gauge?

For $\mathcal{L}u = -\nabla \cdot (D\nabla u) + Cu$ and a constant a ,

$$\mathcal{L}(u + a) = \mathcal{L}u + Ca.$$

Thus Neumann boundary conditions alone do not give a gauge. If $C \neq 0$, shifting the solution changes the equation. Subtracting the mean of the error would hide a real error; for example, the constant field $u_h = 7$ compared with $u = 0$ has error 7, not zero. For $C \equiv 0$ and all-Neumann data, error is compared modulo constants using the visible-cell volume weights. This distinction is independent of source quadrature. The regression test `testEbNeumannSemantics` checks both cases, including `SourceQuad=3` and the well-conditioned scaled problem $D = 10^{-18}$, $C = f = 10^{-14}$, $g = 0$, whose solution is $u = 1$.

E.2 Compatibility after every refinement

Let $V_i = h_{x,i}h_{y,i}$ denote a visible cell's area and A_F a physical boundary face's length. With outward derivative data g_F , the singular conservative equations require

$$S + J = 0, \quad S = \sum_{i \in \mathcal{V}} V_i f_i^{\text{raw}}, \quad J = \sum_{F \subset \partial\Omega} A_F D_F g_F.$$

The implementation computes

$$c_H = \frac{S + J}{\sum_{i \in \mathcal{V}} V_i}, \quad f_i^{\text{projected}} = f_i^{\text{raw}} - c_H.$$

It records c_H in `H.compatibility.sourceShift` and $S + J$ in `H.compatibility.unprojectedDefect`. Each level's `sourceShift` records the constant already removed from its `FTrue`. A fresh level starts at zero. Reprojection restores the old shift in the composite sum and applies the difference between the new and old shifts, so every level represents the same projected source. Without this bookkeeping, successive refinements introduce different constant source perturbations on different levels even when the final global mass defect is small.

For smoothly compatible continuous data, midpoint source and boundary quadrature normally make $c_H = O(h_{\max}^2)$ on a controlled mesh family. For incompatible data it may be $O(1)$: projection then solves a changed problem. Tensor Gauss quadrature improves source integration but is not exact for arbitrary functions and does not improve the separately sampled Neumann boundary flux automatically.

E.3 A multidimensional QIF sign counterexample

Take constant $D > 0$ and a west coarse–fine face away from tangential end exceptions. In the padded fine array, `ebInterpCf` writes

$$Q_{1,3} = \frac{8}{15}Q_{1,2}^c + \frac{2}{3}Q_{2,4} - \frac{1}{5}Q_{3,5}.$$

The operator row for interior value $Q_{2,3}$ contains the term $-DQ_{1,3}/h_x^2$. Therefore the coefficient of the distinct interior unknown $Q_{3,5}$ in that row is

$$+\frac{D}{5h_x^2} > 0.$$

That unknown is not one of the row’s four ordinary nearest neighbors, so no nearest-neighbor diffusion coefficient cancels it. This remains true for constant coefficients and square cells. Consequently the fully ghost-eliminated multidimensional QIF matrix is not, in general, a Z-matrix. A one-dimensional normal-stencil sign argument cannot be applied to this diagonal stencil. Any comparison principle, inverse positivity result, or energy estimate must be justified for the actual composite operator; flux conservation alone does not establish it.

The existing `convergence_theory.tex/pdf` is retained separately. Its multidimensional sign claim and the theorems relying on that claim require further proof work; they should not be treated as verified theorems for this implementation. The measured convergence tables are finite numerical evidence. They neither prove a uniform V-cycle bound nor control arbitrary interface geometry, coefficient contrast, or an unbounded number of patches.

E.4 What the diagnostics establish

The composite residual includes the flux correction and measures algebraic error. At any synchronized state, the conservative mass defect equals minus the volume sum of that residual, up to roundoff. It can therefore be small through cancellation while the maximum residual remains large. Always report both. The error norm adds an independent comparison with a correctly manufactured solution.

The `u1ap` indicator is $\sqrt{(\delta_x^2 u)^2 + (\delta_y^2 u)^2}$, with undivided differences; it is a refinement heuristic, not a rigorous error bound. Proper nesting and minimum-width handling may drop tags. Moreover, the driver only appends a new finest level: it does not revisit dropped tags on older levels or coarsen an existing hierarchy. Inspect the resulting visible-cell coverage when a particular region must be resolved.

E.5 Reproducible command-line checks

From the repository root on the audited macOS installation:

```
1 /Applications/MATLAB_R2026a.app/bin/matlab -batch "maxNumCompThreads(2); cd('EllipticBSAM/tests');  
    unitKernels; runAllTests; auditExtras; stressInterfaces; testNeumannCompat;  
    testEbNeumannSemantics"
```

Use a separate MATLAB path for each solver's test directory: both 2D packages define `mmsCases`. The September 2026 evidence under `Audit/2026-09-10/` distinguishes existing suite results from the new regression tests. Historical all-Neumann, positive-reaction error tables may have removed the error mean; rerun them before comparing absolute errors with the corrected diagnostics.

References

- [1] W. Feng, Z. Guo, J. S. Lowengrub, and S. M. Wise, “A mass-conservative adaptive FAS multigrid solver for cell-centered finite difference methods on block-structured, locally-Cartesian grids,” *Journal of Computational Physics*, 352 (2018), 463–497. doi:[10.1016/j.jcp.2017.09.065](https://doi.org/10.1016/j.jcp.2017.09.065).
- [2] M. J. Berger and I. Rigoutsos, “An algorithm for point clustering and grid generation,” *IEEE Transactions on Systems, Man, and Cybernetics*, 21(5) (1991), 1278–1286. doi:[10.1109/21.120081](https://doi.org/10.1109/21.120081).
- [3] W. L. Briggs, V. E. Henson, and S. F. McCormick, *A Multigrid Tutorial*, 2nd ed., SIAM, Philadelphia, 2000.
- [4] U. Trottenberg, C. W. Oosterlee, and A. Schuller, *Multigrid*, Academic Press, 2001.